

ClinicFlow — Complete System Documentation

Documentation snapshot: 22 September 2026

Product: ClinicFlow multi-clinic appointment and live-queue application

Audience: administrators, doctors, reception staff, patients, project operators, developers, and future maintainers

1. How to use this manual

This manual documents the application that has actually been implemented, rather than presenting the original requirements as completed features. It combines:

- A role-by-role user guide and screen/form reference.
- Backend architecture, business rules, authorization, API behavior, and a database dictionary.
- Setup, administration, development, maintenance, troubleshooting, and release guidance.
- An exact OpenAPI contract appendix for request and response details.

The version-controlled source sections are in `docs/manual/`. The consolidated reference is `docs/ClinicFlow-System-Documentation.md`. A self-contained downloadable HTML edition is provided alongside it, with navigation and print styling.

Status terminology

Term	Meaning
Implemented	Corresponding application code exists; this is not a claim of successful end-to-end acceptance testing.
Verified	A specifically identified check was performed. The scope of that check matters.
Pending / not connected	Configuration or functionality is not available in the current build.
Recommended	A proposed operational practice or future check, not something already implemented or executed.
Development-only	Intended for the preview environment, not a production deployment or real patient information.

Important: This document is a source-based system inventory, not a security certification, compliance opinion, or proof of production readiness. An API contract describes an interface; actual implementation details and limitations are discussed separately.

2. Purpose and boundaries

ClinicFlow coordinates multiple clinics and branches, doctors' availability, appointment bookings, token allocation, front-desk processing, and live queue information.

The primary business flow is:



The app is not an electronic medical record, prescribing platform, billing system, or telemedicine service. No such capability should be inferred from the existence of patient or appointment records.

Separate authenticated roles

The application does **not** provide a role-switching dropdown. Each authenticated identity is mapped to a stored application role.

Role	General responsibility
Super Admin	Platform-wide administration, users, master data, configuration, and oversight.
Clinic Admin	Administration within assigned clinic scope, with no platform-level privilege grant.
Doctor	Own professional profile, assigned/owned care locations, availability, appointments, relevant patients, and queue operations.
Receptionist	Assigned-location patient registration, appointment booking, check-in, and queue management.
Patient	Own profile, booking, appointments, cancellation where allowed, and privacy-limited queue information.

Detailed permissions and differences between visible navigation and server enforcement appear in the following sections. Hiding a menu item is not the security boundary; the API must authorize the request.

3. System at a glance

```
Browser
└─ React application + Clerk sign-in + query cache
    └─ Clerk identity/session service
        └─ Generated REST client → Express API
            ├── Identity-to-role mapping
            ├── Scope and input validation
            ├── Availability / appointment / queue logic
            ├── PostgreSQL transactions and audit records
            └─ OTP delivery adapter
                ├── Development code response
                └─ Twilio (not connected in this snapshot)
```

- **Frontend:** React and TypeScript, Vite, Wouter routing, TanStack Query, generated API clients, and Clerk authentication components.
- **Backend:** Node.js, Express, TypeScript, request validation, authorization, and transaction-based business operations.
- **Storage:** PostgreSQL with Drizzle; core identities, relationships, statuses, tokens, and scope keys are relational. Some optional fields and configuration are stored as JSONB.
- **Contract:** `lib/api-spec/openapi.yaml`, with generated client and validation packages.
- **Live queue:** periodic polling, not a WebSocket push service.
- **QR codes:** revocable public booking references, not encoded patient records.

4. What has actually been checked

During the initial implementation, type checks were reported passing for the shared libraries, API, scripts, and frontend. API and frontend workflows were started successfully. A screenshot confirmed that the public landing page renders.

Subsequently, the preview-account provisioning script passed its TypeScript check and successfully created separate development identities and application profiles for the five roles.

These checks do **not** prove:

- Successful sign-in and complete navigation for every role.
- End-to-end multi-account availability → booking → token → staff queue → patient queue behavior.
- Isolation under malicious cross-clinic requests.
- Correctness under concurrent bookings and queue operations.
- Production SMS delivery, production authentication configuration, or recovery workflows.
- Accessibility compliance, regulatory compliance, backup restoration, or production load capacity.

No full multi-account acceptance test, security certification, or concurrency test has been completed as part of the documented first build. Later sections provide an acceptance checklist, explicitly as work still to perform.

5. Current environment and external-service status

Preview accounts

Five explicit test accounts were created at the user's request after the initial build: Super Admin, Clinic Admin, Doctor, Receptionist, and Patient. They are genuine development authentication accounts with fixed database roles, not a simulated login bypass.

Passwords are deliberately excluded from this manual and from source-controlled documentation. They were supplied separately in the conversation. Do not copy them into source files, public documentation, production configuration, or screenshots.

The test emails use Clerk's `+clerk_test` convention. **For these development test emails only, the email verification code is `424242`**, including the new-device email challenge. This is not the application's mobile OTP and must not be used as a production verification mechanism.

Account creation did not automatically create clinics, branches, doctor schedules, appointments, or clinic assignments. Staff with no assignments will have limited or empty scoped data. Begin with Super Admin to create and assign the desired setup.

Two separate verification systems

System	Purpose	Current behavior
Clerk authentication	Sign-in identity, passwords, email verification, and sessions	Configured for development preview. Test-email challenges use the development convention above.
ClinicFlow mobile OTP	Verify the authenticated user's mobile number for applicable booking policy	Explicit development provider is available; it displays a development code and sends no SMS.

Twilio was proposed but the connection prompt was dismissed. **No real SMS provider is connected in this snapshot.** Production must fail rather than silently accept the development provider.

Known release boundaries

- Broad notification delivery beyond the implemented OTP adapter and Clerk invitations is not connected.
- A stored session-timeout setting does not change Clerk's actual session lifetime.

- Collection filtering and pagination are largely performed in the API process after reading persisted rows; larger deployments need further database-side optimization.
- Weekly availability supports one session and one optional break per relevant branch/day; overnight sessions are not supported.
- Cross-branch scheduling across different time zones uses conservative overlap checks.
- External identity invitations and local database writes are not one atomic transaction.
- No separate visual wireframe was supplied; the written specification was used as the journey reference.
- Do not put real patient information into the system before appropriate acceptance, security, operational, and legal review.

The later chapters identify finer-grained limitations at the relevant feature.

6. Document maintenance

Regenerate the consolidated document after updating its source chapters or the API contract. Treat generated API packages as derived code, not the place to redefine interfaces.

When a feature changes, update its user workflow, permission description, API/schema details, operational instructions, and verification status together. Never change a status from “implemented” to “verified” without naming the check actually performed.

This manual intentionally contains no credentials, environment secret values, live patient records, or claims about an unpublished production environment.

ClinicFlow frontend user guide

Basis and status of this guide. This guide documents the frontend as implemented in the current source, not an aspirational product design. “Wired” means the UI calls a generated API client or Clerk flow in code. It does **not** mean that the feature was exercised against a configured deployment. Items marked **Limitation / unverified** either have no frontend control, depend on deployment configuration or third-party delivery, or were not runtime-tested during this documentation pass.

1. What ClinicFlow provides

ClinicFlow has:

- a public marketing home page;
- Clerk-hosted sign-in, sign-up, and account-recovery experiences;
- first-time patient or doctor onboarding;
- public QR booking links that hand off to authenticated booking;
- role-specific workspaces for platform administrators, clinic administrators, doctors, receptionists, and patients;
- appointment booking, status transitions, and live queue views;
- administration of clinics, branches, doctors, patients, users, master values, schedules, exceptions, and booking QR codes;
- reports, CSV export, audit records, platform settings, and self-service profiles.

The frontend sends API calls through `/api`. The generated contract says data is persisted and authorization-scoped: super administrators can access all records; clinic administrators and receptionists are assignment-scoped; doctors access their own/relevant resources; patients access themselves. UI visibility is an additional convenience, not the security boundary.

2. Routes, redirects, and access

2.1 Public and authentication routes

Route	Behavior
/	Public home when signed out. A signed-in visitor is immediately redirected to <code>/onboarding</code> .
<code>/sign-in</code> and nested Clerk paths	Clerk sign-in UI. On success, forcibly redirects to <code>/onboarding</code> .
<code>/sign-up</code> and nested Clerk paths	Clerk sign-up UI. On success, forcibly redirects to <code>/onboarding</code> .
<code>/login</code>	Alias that redirects to <code>/sign-in</code> .
<code>/register</code>	Alias that redirects to <code>/sign-up</code> .
<code>/forgot-password</code>	Redirects to <code>/sign-in</code> ; recovery itself is selected inside Clerk's sign-in UI. There is no standalone ClinicFlow recovery form.
<code>/register-doctor</code>	Stores a browser-session <code>doctor</code> onboarding intent and redirects to <code>/sign-up</code> .
<code>/onboarding</code>	Connects an authenticated Clerk identity to a ClinicFlow patient or doctor profile.
<code>/book/:reference</code>	Resolves a public booking QR reference. The clinic/branch/doctor context may be preselected and locked. Authentication is required before booking.
Any unmatched route	"Page not found" with a Return home link.

The public home's header has **How it works**, **For clinics**, **Sign in**, and **Get started**. The main calls to action are **Book an appointment** (patient sign-up) and **I'm a healthcare provider / Join as a doctor** (doctor-intent sign-up).

2.2 Workspace route model

All workspace routes are guarded. While Clerk or the current ClinicFlow identity is loading, the page says **Preparing your workspace....** Signed-out users go to `/login`. A missing ClinicFlow user or `needsOnboarding` identity goes to `/onboarding`. An identity error is shown with **Try again**.

The API roles `superAdmin` and `clinicAdmin` both use the `/admin/...` URL family and admin shell. If a user enters the wrong role family, the guard redirects to that user's actual dashboard. Visiting `/admin`, `/doctor`, `/receptionist`, or `/patient` redirects to that role's `/dashboard`.

All registered workspace routes are:

- `/admin/dashboard`, `/admin/clinics`, `/admin/branches`, `/admin/doctors`, `/admin/users`, `/admin/patients`, `/admin/masters`, `/admin/appointments`, `/admin/queue`, `/admin/reports`, `/admin/settings`, `/admin/audit`, `/admin/qrs`, `/admin/book`

- `/doctor/dashboard` , `/doctor/profile` , `/doctor/clinics` , `/doctor/branches` ,
`/doctor/availability` , `/doctor/exceptions` , `/doctor/appointments` , `/doctor/queue` ,
`/doctor/patients` , `/doctor/qrs` , `/doctor/book`
- `/receptionist/dashboard` , `/receptionist/appointments` , `/receptionist/queue` ,
`/receptionist/patients` , `/receptionist/book`
- `/patient/dashboard` , `/patient/book` , `/patient/appointments` , `/patient/queue` ,
`/patient/profile`

2.3 Navigation visibility by role

The sidebar order and visibility are:

Role	Visible sidebar items
Super administrator	Overview; Appointments; Live queue; Clinics; Branches; Doctors; Patients; Users; Booking QR codes; Reports; Master data; Audit log; Settings
Clinic administrator	Overview; Appointments; Live queue; Clinics; Branches; Doctors; Patients; Booking QR codes; Reports
Doctor	Overview; Appointments; Live queue; Patients; Clinics; Branches; Weekly schedule; Date exceptions; Booking QR codes; Profile
Receptionist	Overview; Appointments; Live queue; Patients
Patient	Overview; Book appointment; Appointments; Live queue; Profile

Clinic administrators are explicitly redirected away from `/admin/users` , `/admin/masters` ,
`/admin/settings` , and `/admin/audit` , even if those URLs are entered manually.

Book route discoverability. Admin, doctor, and receptionist sidebars do not contain **Book appointment**, but the route is registered. Most page headings show a **Book appointment** shortcut; it is suppressed only on Book, Profile, and Settings. Patients also have Book in their sidebar. Thus staff booking is wired but often reached from a page-heading shortcut or dashboard card rather than sidebar navigation.

Limitation / unverified: frontend routes and menus are not a complete statement of server permission. For example, a doctor can see Clinics and Branches, but the API can reject edits outside owned scope. Receptionist QR write permission exists in the API implementation but no receptionist QR route or navigation item exists in this frontend.

3. Authentication, onboarding, and session behavior

Sign-in and sign-up

The auth layout uses Clerk components and includes a back-to-home link. Sign-in links to sign-up and vice versa. Clerk controls credential fields, verification, forgot-password steps, session duration, and related validation; those exact fields vary with the configured Clerk instance and are not defined by ClinicFlow source.

After sign-in/sign-up, the user is sent through `/onboarding`. If the ClinicFlow identity already exists, onboarding redirects to:

- `/admin/dashboard` for `superAdmin` or `clinicAdmin`;
- `/<role>/dashboard` for doctor, receptionist, or patient; or
- `/book/:reference` if a QR reference remains in session storage.

When the Clerk user changes, the React Query cache is cleared. Sign out also clears cached queries and calls Clerk sign-out, redirecting to the application base URL.

First-time onboarding

The onboarding form contains:

- **Full name** — required; initialized from the Clerk full name when available.
- **Mobile** — optional telephone input.
- **I consent to the use of my information for appointment management** — required checkbox.
- **Complete my profile** submit button, shown as disabled with **Saving...** semantics while the request is pending.

Normal registration defaults to the **patient** intent. Entering through `/register-doctor` sets a session-only doctor intent. Successful onboarding clears that intent.

Limitations / unverified:

- Doctor intent is stored in `sessionStorage`, so it is browser-tab/session dependent. Entering `/sign-up` directly creates the patient intent.
- The onboarding screen displays API errors but does not present a dedicated recovery path beyond returning to Clerk sign-in.
- Clerk session timeout is not controlled by the ClinicFlow settings screen.

QR handoff during authentication

Opening `/book/:reference` stores the reference in session storage and resolves it before authentication. A signed-out user sees the resolved clinic context and **Sign in to book**. After sign-in and any required onboarding, the browser returns to the QR booking route. A successful booking removes the stored QR reference.

4. Shared workspace behavior

The desktop workspace has a fixed sidebar, current-page breadcrumb, “Secure workspace” marker, initials avatar, name/role, and sign-out control. On small screens the sidebar becomes a toggleable drawer. Tables scroll horizontally; two-column forms collapse to one column; QR cards collapse to one column.

Page headings show the current localized weekday/month/day. Dashboard greetings vary by local browser hour. The footer states that all times follow the configured clinic timezone, although browser-local formatting is used for several displayed timestamps.

Errors, loading, and empty states

- API errors generally appear inline in a red alert box using the API error message.
- Resource lists show **Loading [resource]...** while loading and **No [resource] yet** when empty.
- Dashboard statistic values show `...` during load and `–` when absent; its appointment panel shows **Loading appointments....**
- Appointment lists show **Loading appointments....**
- Queue shows **Updating live queue...** only after both doctor and branch are selected. Before a valid selection/data response it shows **No selected queue yet**.
- Booking shows **Checking availability...** while availability loads and a “No session.../server reason when unavailable. An empty public clinic result has its own empty state.
- Public QR resolution shows **Finding your clinic....**
- Onboarding shows **Connecting your account....**
- Profile shows **Loading profile...** until data arrives.
- Settings shows its integration notice immediately but has no separate loading skeleton for the settings form.
- Reports has no explicit loading indicator: before/without rows it displays the empty report-results state.
- QR image-generation errors, QR regeneration errors, form-save errors, delete/deactivate errors, OTP errors, and appointment-transition errors are displayed inline.
- A top-level React error boundary is present for caught rendering failures.

Generic search and pagination

Clinics, branches, doctors, patients, users, master data, and audit logs have a search box. Search is sent server-side as the user types, with no submit button or debounce, and resets the page to 1. Weekly schedules, date exceptions, and QR codes do not offer search.

Generic resource lists request 10 records per page. The footer shows total records and current page.

Previous is disabled on page 1; **Next** is disabled when `page × 10 >= total`. There is no page-number picker, page-size selector, or sort control.

Appointments have separate server-side search, date, and status filters, also with 10 rows per page.

Changing any filter resets page 1. The status dropdown contains All statuses plus: `booked`, `checkedIn`, `waiting`, `called`, `inConsultation`, `completed`, `noShow`, and `cancelled`. Search is described as patient/reference search.

Lookup dropdown behavior

Relation fields are native selects populated from API records. Multi-value fields (keys ending in `Ids`) are native multi-selects and instruct desktop users to hold Ctrl/Command. Lookup loading is indicated in the first option; lookup errors appear below the control.

Lookup loaders fetch all available records in pages of 100, fetching up to four later pages concurrently per batch. Generic editor relation lists are not dynamically narrowed by earlier choices in the same form. In particular, a Branch relation can show branches beyond a selected Clinic; invalid combinations may be rejected by the API.

City/state/pincode/country/area text fields use datalist suggestions from active matching master values. They remain free-text inputs; selecting a suggestion is not mandatory.

5. Overview dashboard

All roles use the same dashboard layout, with API data scoped to their identity:

1. Four statistic cards:
 - Today's appointments
 - Waiting in queue
 - Completed visits
 - Average wait in minutes
2. A recent appointments table: labeled **Your appointments** for patients and **Today's appointments** for staff, with **View all**.
3. A booking card linking to the role's Book route.
4. **Workspace activity**, showing summary, actor, and localized timestamp, or an empty state.

The dashboard appointment table uses the same appointment actions as the full list. The API currently returns recent audit activity only to administrator roles, so other roles normally see an empty activity state.

6. Appointment list and appointment actions

Each appointment row displays:

- patient name and booking reference;
- doctor, clinic, and branch;
- date and token;
- status;
- only the next actions returned as allowed by the API.

Possible action labels are **Check in**, **Enqueue**, **Call**, **Start**, **Complete**, **No show**, **Requeue**, and **Cancel**. The current server state flow is:

```
booked → checkedIn → waiting → called → inConsultation → completed
```

Additional flows are:

- `waiting` or `called` → `noShow` ;
- `noShow` → `waiting` via requeue;
- `booked` , `checkedIn` , or `waiting` → `cancelled` .

Cancel and No show open a browser prompt requesting a reason. Cancelling the prompt aborts the action; submitting an empty reason is permitted by the frontend. Buttons are disabled while a transition is pending. The frontend includes the currently displayed status as `expectedStatus` , allowing the API to reject stale transitions. Patients receive only Cancel for their own eligible booking; staff actions are API-scoped.

Appointments poll automatically every **30,000 ms (30 seconds)**. Dashboard recent appointments do not set their own polling interval. Actions invalidate cached queries, causing affected data to refetch according to React Query behavior.

Limitations / unverified:

- There is no appointment details page, history/timeline UI, manual refresh button, direct appointment editor, or standalone appointment-delete control.
- Queue actions may be server-restricted to the appointment date; cancellation may be rejected after the configured cutoff.
- The reason prompt is a basic browser dialog, not a controlled form or cancellation-reason dropdown.

7. Booking an appointment

Booking is a three-step workflow. A client-generated request ID is reused for the mounted booking form to support idempotent submission.

Step 1 — Choose your care

Fields:

- **Clinic** — public clinic records.
- **Branch** — disabled until a clinic is selected.
- **Doctor** — disabled until a branch is selected.
- **Visit date** — defaults to today and cannot select an earlier date in the browser.

Changing clinic clears branch and doctor. Changing branch clears doctor. Public branch and doctor queries are filtered using the current clinic/branch values. Availability is fetched only when doctor, branch, and date are present.

When available, the screen shows remaining places, session start/end, timezone, and optional break. It explains that appointments are queue-based and do not guarantee an exact consultation time. Continue is disabled while availability is fetching, when unavailable, or when fewer than one token remains.

With a QR booking context, any encoded clinic, branch, and doctor selections are prefilled and disabled. A QR may encode only clinic, clinic+branch, or clinic+branch+doctor.

Step 2 — Your details

For a patient:

- the visit is fixed to the signed-in patient profile;
- **Mobile number** is initialized from the user profile;
- **Send code** requests an OTP;
- after an OTP challenge, the provider and localized expiry time are displayed;
- **Verification code** accepts numeric input;
- **Verify** submits the challenge and code;
- successful verification shows **Mobile verified**;
- a development-only code is shown only if the API explicitly returns one in development mode;
- **Notes for your visit** is optional and capped at 1,000 characters.

The expected mobile format shown to users is international E.164 style (leading  ; API contract permits 8–15 digits after it). Verification codes are 4–8 digits per the API contract.

For staff:

- **Patient** selects from all authorized patients;
- **Register a new patient** toggles an inline patient registration form;
- successful registration selects the new patient and closes the form;
- **Notes for your visit** is optional and capped at 1,000 characters.

The staff inline patient form contains all patient fields listed in Patients, prefilled with the selected clinic and branch.

Important OTP limitation: the frontend does not require `verify.data.verified` before allowing Review or Confirm. Whether verification is mandatory is ultimately enforced by API/platform configuration. The UI also does not implement a visible resend countdown from `resendAfterSeconds` ; Send code is disabled only during the pending request or when mobile is empty.

Step 3 — Review & book

The review shows clinic/location, doctor, date/session window, and patient. The user must check:

I consent to appointment management and the clinic's booking policy.

The confirm button is disabled without consent or during submission and changes to **Confirming....** Availability is checked again by the API. Booking source is submitted as:

- `qr` from a resolved QR link;
- `online` for a patient's ordinary booking;
- `walkIn` for staff booking.

Although `phone` exists in the API contract, this frontend never selects it.

Confirmation

Success shows doctor, clinic, branch, assigned token, date, booking reference, and the reminder that booking and check-in are separate. The live Queue component is embedded below and initialized to the newly booked appointment.

There is no email/SMS confirmation control, calendar-file download, payment step, or print-confirmation control in the current frontend.

8. Live queue

The queue API is polled every **30,000 ms (30 seconds)** whenever doctor and branch are selected. The screen also prints **Refreshes every 30 seconds** and shows the server-provided last-updated time formatted in the browser locale.

Patient queue

Patients choose **Your appointment** from authorized appointments for the selected queue date. Each option shows token, doctor, and branch. Selecting one sets appointment, doctor, and branch. Patients can also change Queue date.

The result shows:

- Now serving token, or `–` / “No active consultation”;
- Up next token, or `–`;
- waiting and completed counts;
- if returned, **Your token**, patients ahead, estimated wait in minutes, and status.

The API contract omits the full queue `entries` list for patients. The frontend therefore gives a personal position/status rather than exposing other patients.

Staff queue

Administrators, doctors, and receptionists select Doctor, Branch, and Queue date. A doctor defaults to their own doctor ID; an appointment confirmation can also initialize all queue filters.

Staff additionally see:

- **Call next patient**, disabled while pending;
- the queue entries table, including names/references and allowed appointment actions.

The API picks/advances the next patient and can reject invalid scope, date, or active-consultation conditions.

Limitations:

- Doctor and Branch are independent all-record dropdowns in the queue; the frontend does not filter branch by doctor or validate the pairing before requesting.
- Changing a patient's date does not explicitly clear a previously selected appointment.
- There is no pause queue, manual reorder, priority editor, sound/notification, or dedicated manual refresh.
- The UI text uses a fixed 30 seconds rather than reading `pollIntervalSeconds` dynamically, though the current contract also fixes it at 30.

9. Resource administration

All resource pages use the same patterns:

- table rows are the only read/list view; there is no separate details view;

- **Add** opens a modal with a create form;
- pencil opens the same modal with existing data;
- trash asks **Delete or deactivate this record? Records with history are preserved.**
- the wired DELETE API deactivates records rather than providing a destructive frontend hard-delete flow;
- create/edit/deactivate errors are shown inline;
- successful create/edit closes the modal and refreshes queries;
- successful deactivate refreshes queries;
- required fields show an asterisk and browser/form-level "Please complete this field."

Not every underlying API rule is reproduced in the form. Server validation and authorization remain authoritative.

Clinics

Visible to: super administrator, clinic administrator, doctor.

Table: Name, Code, City, Phone, Status.

Create/edit fields:

- Name *; Address *
- City, State, Pincode, Country, Area (free text with active master-value suggestions)
- Phone; Email; Description
- Clinic type (master-value dropdown)
- Category (clinic-category master-value dropdown)
- Status: Active or Inactive

Create, edit, and deactivate are wired. The API may prevent a clinic administrator from creating a clinic and may restrict doctors to owned clinics.

Branches

Visible to: super administrator, clinic administrator, doctor.

Table: Name, Clinic name, City, Timezone, Status.

Create/edit fields:

- Clinic *
- Name *; Address *
- City, State, Pincode (free text with suggestions)
- Phone; Timezone

- Status: Active or Inactive

Create, edit, and deactivate are wired. Clinic and branch scope/ownership are server-validated.

Doctors

Visible to: super administrator and clinic administrator. Doctors manage themselves through Profile rather than the Doctors list.

Table: Full name, Specialization name, Email, Mobile, Status.

Create/edit fields:

- Full name *; Mobile *; Email *
- Date of birth; Gender: Female, Male, Other, Prefer not to say
- Photo URL
- Specialization
- Registration number; Experience years; Consultation fee
- About
- Clinics (multi-select); Branches (multi-select)
- Qualifications (multi-select)
- Languages (comma-separated; trimmed into an array)
- Status: Active or Inactive

Creating a doctor is wired to account/profile provisioning and therefore depends on Clerk invitation/configuration. No credentials are entered or displayed in this form. Edit and deactivate are wired.

Patients

Visible to: super administrator, clinic administrator, doctor, receptionist. Patients use Profile rather than the list.

Table: Full name, Code, Mobile, Gender, Mobile verified.

Create/edit fields:

- Full name *; Mobile *
- Email; Date of birth
- Gender: Female, Male, Other, Prefer not to say
- Age; Address
- Emergency contact name; Emergency contact phone
- Clinic; Branch

- Status: Active or Inactive

Create, edit, and deactivate are wired for staff subject to API scope. The table renders Mobile verified as Yes/No. Changing mobile can reset verification server-side.

Users

Visible to: super administrator only in this frontend. Clinic administrators are explicitly blocked from the page despite some assignment-scoped API support.

Table: Full name, Email, Role, Status.

Create/edit fields:

- Full name *: Email *
- Mobile
- Role *: Doctor, Receptionist, Clinic admin, Patient
- Clinics (multi-select); Branches (multi-select)
- Status: Active or Inactive

Super administrator is not offered as a create/edit role in the UI. Account creation is wired to identity matching/invitation and can fail if Clerk is unavailable or an identity/email conflicts. Existing roles may be immutable server-side even though the edit form displays Role. Deactivation is wired; server protections can prevent deactivating the final active super administrator.

Account recovery

The Users page includes an **Account recovery** panel:

1. Select a user from the **current page's results**.
2. Click **Request recovery instructions**.
3. On success, read the returned message and use **Open secure sign-in and select Forgot password**.

The request is audited but explicitly does **not** itself send a recovery email. The actual password recovery is Clerk's secure sign-in flow. No password, reset token, or secret is displayed or edited by ClinicFlow.

Master data

Visible to: super administrator only.

Table: Name, Category, Code, Status.

Create/edit fields:

- Category *: Country, State, City, Area, Pincode, Clinic type, Clinic category, Clinic status, Specialization, Qualification, Department, Consultation type, Appointment status, Appointment type, Booking source, Cancellation reason, Queue status, Token prefix, Queue type, Queue priority, User role, User status
- Name *; Code *
- Parent (master-value dropdown)
- Sort order
- Status: Active or Inactive

Create, edit, and deactivate are wired. Some categories have governed codes and the API may reject unsupported or changed codes.

Weekly schedules (/doctor/availability)

Visible to: doctor. A doctor-linked identity opens new schedules with its Doctor preselected and `isOpen` true.

Table: Doctor name, Branch name, Day of week, Start time, End time, Max tokens.

Create/edit fields:

- Doctor *; Clinic *; Branch *
- Day of week *: Sunday through Saturday
- Is open
- Start time *; End time *
- Break start; Break end
- Timezone
- Token prefix *
- Max tokens *; Consultation minutes *; Buffer minutes
- Queue mode: Mixed, Appointments only, Walk ins only
- Queue open time; Queue close time

The modal warns: one session and optional break per branch/day; overnight sessions are unsupported. Create, edit, and deactivate are wired. API validation handles time validity and overlapping sessions.

The list has pagination but no search.

Date exceptions (/doctor/exceptions)

Visible to: doctor.

Table: Date, Reason, Is closed, Start time, End time.

Create/edit fields:

- Doctor *; Branch *
- Date *
- Is closed
- Reason *
- Start time; End time
- Break start; Break end
- Max tokens

Use Is closed for closures; for an override, enter replacement timing/capacity. Create, edit, and deactivate are wired. The API can require an underlying weekly schedule and reject overlaps.

The list has pagination but no search.

Audit log

Visible to: super administrator only in this frontend.

Table: Created at, Actor name, Action, Entity type, Summary.

Audit is read-only: there is search and pagination, but no create, edit, deactivate, details, date filter, or export control. Timestamps are rendered as returned in the generic table (not explicitly locale-formatted here).

10. Booking QR codes

Visible to: super administrator, clinic administrator, and doctor.

Table: Name, Reference, Status. It has pagination but no search.

Generate

Add QR code opens fields:

- Name *
- Clinic *
- Branch (optional)
- Doctor (optional)
- Status: Active or Inactive

Saving is wired to QR creation. The API generates the public reference; the user never types the reference. Branch/doctor context is optional, enabling progressively less specific links.

View and download

Every active QR in the current result page is also rendered as a card below the table. The browser generates a 400-pixel data-URL QR image for the full `/book/:reference` URL. The card shows the QR image, name, clickable public URL (new tab), and **Download PNG**, which downloads `[QR name] -qr.png`.

Inactive QR rows remain visible in the table but do not render cards.

Regenerate

Regenerate reference asks for confirmation that printed copies will stop working. On confirmation it calls the regeneration API and refreshes queries. The resulting reference/link and QR image change. Regeneration errors are shown on the card.

Deactivate

The table's trash action uses the generic deactivate confirmation. The wired API marks the QR inactive; after refresh it disappears from the active-card grid. Edit can also set Status to Inactive.

Print status

Limitation: there is no QR-specific Print button, print stylesheet, or `window.print()` action in the current frontend. Users may download the PNG and print it using browser/operating-system tools. Regeneration invalidates prior printed/downloaded codes by replacing their reference.

11. Reports and CSV export

Visible to: super administrator and clinic administrator in the frontend.

Filters:

- From date — defaults to today
- To date — defaults to today
- Group by — Date, Clinic, or Doctor

Results columns:

- Group
- Appointments

- Completed
- Cancelled
- No-show
- Registrations
- Average wait (minutes)

Changing a filter directly changes the report query; there is no Apply button. An API error is shown inline. No rows produces **No report results yet**.

Export CSV is enabled only when rows exist. It creates a local `clinicflow-report.csv` containing `label`, `appointments`, `completed`, `cancelled`, `noShow`, `registrations`, and `averageWaitMinutes`. Values are quoted and embedded quotes escaped. Export uses the currently loaded grouped rows, not a separate export endpoint.

Limitations: no charts, print report, PDF/XLSX export, clinic/branch filter, scheduled report, custom filename, or explicit loading indicator. Checked-in/waiting and average consultation metrics may exist in API data but are not shown/exported by this UI.

12. Platform settings

Visible to: super administrator only.

The settings form is loaded from and saved to the API. Fields shown:

- Platform name
- Support email
- Support phone
- Timezone
- Booking horizon days
- Cancellation cutoff minutes
- Require mobile verification
- OTP expiry seconds
- OTP max attempts
- Terms URL
- Privacy URL

Successful save shows **Settings saved**. The notice displays whether an SMS provider is configured and states Queue refresh: 30 seconds.

Explicit boundaries:

- General notifications are not connected.

- Session timeout is managed by Clerk; settings here do not change the authentication session.
- Although the API settings schema includes session timeout, notifications, logo URL, and primary color, this frontend does not expose those fields.
- The form provides required-field checks only where configured; numeric ranges and valid timezone/URL/email rules may be rejected by the API.

13. Profiles

Patient profile

Fields:

- Full name *
- Mobile *
- Email
- Date of birth
- Gender: Female, Male, Other, Prefer not to say
- Age
- Address
- Emergency contact name
- Emergency contact phone

Clinic, Branch, and Status are deliberately omitted from self-edit. Saving is wired to the patient update API. Changing mobile resets verification server-side.

Doctor profile

Fields:

- Full name *
- Mobile *
- Email *
- Date of birth
- Gender
- Photo URL
- Specialization
- Registration number
- Experience years

- Consultation fee
- About
- Clinics (multi-select)
- Branches (multi-select)
- Qualifications (multi-select)
- Languages (comma-separated)

Status is omitted. Saving is wired to the doctor update API; server rules can prevent doctors from adding assignments.

Fallback user profile

If an identity has neither doctor nor patient profile, fields are Full name *, Mobile, and Photo URL, saved through the current-user profile endpoint. The registered role routes normally expose Profile only to doctor and patient.

All profile types show a note that mobile changes reset verification, inline errors, and a **Your profile has been saved** success notice.

14. Feature status and known frontend limitations

Clearly wired in source

- Clerk sign-in, sign-up, sign-out, and handoff to Clerk recovery.
- Identity-aware redirects, onboarding, and patient/doctor intent.
- Scoped dashboard API.
- Resource list/create/edit/deactivate calls.
- Appointment list, filters, 30-second polling, and state transitions.
- Three-step booking, availability checks, idempotency key, patient registration, OTP request/verify, consent, confirmation.
- Public QR resolution and auth return path.
- Queue selection, patient-own entry, staff entries, Call next, and 30-second polling.
- QR create/edit/deactivate, client-side image generation, PNG download, and reference regeneration.
- Reports and browser-side CSV export.
- Settings read/update.
- Audit list.
- User recovery-instruction request.

- Doctor/patient/current-user profile update.

Pending, absent, or deployment-dependent

- No runtime verification was performed for this documentation; successful delivery depends on API, database, Clerk, and optional SMS configuration.
- No frontend-specific print function for QR codes, appointments, queue, confirmations, or reports.
- No general notification center, email/SMS booking confirmation control, or queue push notifications.
- SMS OTP delivery is deployment-dependent. A development provider may return a development-only code; this is not delivered SMS.
- OTP completion is not a frontend gate to booking; API settings/enforcement decide whether verification is required.
- No read-only detail pages for resources or appointments; tables are the read presentation.
- No sorting controls, adjustable page sizes, numbered-page jump, saved filters, or search debounce.
- No hard-delete user workflow. The generic action and server behavior are deactivation-oriented.
- No dedicated reactivate action; records with editable Status can be set Active in Edit if allowed, while resources without a Status field may require another operational path.
- No QR card for inactive records and no frontend preview of a QR before creation.
- No report loading state or report auto-refresh interval.
- Only Appointments and Live queue explicitly poll, each every exactly 30 seconds. Other screens use normal query caching/refetch/invalidation; global query defaults are one retry and 15-second stale time.
- Browser locale is used for headings and several timestamps, while clinic-configured timezone governs server scheduling. Display-zone differences should be considered.
- Native multi-select interaction is desktop-oriented; the Ctrl/Command instruction is not tailored for touch devices.

15. Source map

Primary implementation references:

- Route registration, role-route guards, public home, Clerk auth, and redirects: `artifacts/clinicflow/src/App.tsx` (especially lines 10–17, 25–65).
- Onboarding, public QR handoff, role navigation, dashboards, appointments, booking, queue, profiles, settings, and reports: `artifacts/clinicflow/src/clinic.tsx` (lines 10–95).

- Resource definitions, every editor field, generic CRUD/search/pagination, user recovery, QR rendering/download/regeneration, profile/settings field lists: `artifacts/clinicflow/src/resources.tsx` (lines 10–86).
- Responsive behavior and visible state styling: `artifacts/clinicflow/src/index.css` (especially lines 6–10).
- Generated roles, statuses, field constraints, queue contract, QR context, reports, and settings types: `lib/api-client-react/src/generated/api.schemas.ts` (notably lines 28–69, 78–169, 172–683).
- API contract scope/error/polling summary: `lib/api-client-react/src/generated/api.schemas.ts` lines 1–11.
- API error-message behavior: `lib/api-client-react/src/custom-fetch.ts` lines 151–199 and 325–370.

The source pointers are included so operators and testers can distinguish frontend behavior from assumptions and re-check this guide when the implementation changes.

ClinicFlow backend manual

This document describes the backend that is present in this repository. It is an implementation reference, not a product roadmap. The HTTP implementation is in `artifacts/api-server/src` ; the contract used to generate clients and Zod validators is `lib/api-spec/openapi.yaml` ; the database model and migrations are in `lib/db/src/schema/core.ts` and `lib/db/drizzle` .

1. Runtime architecture

- The service is an Express application mounted under `/api` . `PORT` is mandatory and must be a positive number. [source: `artifacts/api-server/src/index.ts:4-24` ; `artifacts/api-server/src/app.ts:13-64`]
- Request processing, in order, is: Pino HTTP logging; Helmet (CSP disabled); production Clerk frontend-API proxy; CORS; JSON (128 KiB limit) and URL-encoded parsing; Clerk authentication; global API rate limiting; mutation-origin checking; route dispatch; API 404; common error handling. [source: `artifacts/api-server/src/app.ts:15-64`]
- PostgreSQL access uses Drizzle. Most records split authorization/indexing fields into relational columns and optional/domain fields into a non-null JSONB `data` object. Store reads flatten `data` into the returned object and then overlay relational columns, so a relational column wins if the same key exists in JSON. [source: `artifacts/api-server/src/lib/store.ts:7-23` ; `lib/db/src/schema/core.ts:4-75`]
- Collection operations currently select whole tables, enrich and authorize rows, filter, sort, and paginate in application memory. Dashboard and reports are likewise in-memory aggregates rather than SQL aggregates. [source: `artifacts/api-server/src/lib/store.ts:12,37-52` ; `artifacts/api-server/src/routes/resources.ts:166-172` ; `artifacts/api-server/src/routes/reporting.ts:16-43`]
- IDs are UUID strings generated with `crypto.randomUUID()` , except public QR references, which are 32 random bytes encoded as base64url. Appointment human references use 16 uppercase hexadecimal characters after `CF-` ; generated clinic/branch/doctor/patient codes use the first eight characters of an ID. [source: `artifacts/api-server/src/lib/store.ts:1-6` ; `artifacts/api-server/src/routes/resources.ts:124-141` ; `artifacts/api-server/src/routes/appointments.ts:60-64`]

HTTP and error behavior

- All routes below are relative to `/api` .

- JSON errors normally have `{error, code}`. Assertions default to code `REQUEST_FAILED`. Zod failures are HTTP 400/code `VALIDATION_ERROR`; PostgreSQL unique, foreign-key, and check failures (`23505`, `23503`, `23514`) become generic HTTP 409; unexpected failures become HTTP 500 with the public message `Service unavailable. Please retry.`. Unknown API routes return HTTP 404 `{error:"API route not found"}` without the common `code` field. [source: `artifacts/api-server/src/lib/http.ts:3-12,37-42`; `artifacts/api-server/src/app.ts:63-64`]
- Body dates named `date`, `from`, `to`, or `dateOfBirth` must be real `YYYY-MM-DD` dates. A body field named `mobile`, where present and truthy, must match E.164-like `^\+[1-9][0-9]{7,14}$`. Query `date` / `from` / `to` receive the same calendar validation and are normalized back to strings after generated-schema parsing. [source: `artifacts/api-server/src/lib/http.ts:9-35`]
- Generated Zod object schemas strip unknown object properties. The resource implementation uses each resource's **create** schema for both POST and PATCH, so PATCH requests must supply every field marked required below; they are not generic partial updates. [source: `artifacts/api-server/src/routes/resources.ts:14-24,177-183`; generated validators in `lib/api-zod/src/generated/api.ts`]
- Pagination defaults to page 1 and page size 20; schema maximum page size is 100. Results are `{items,total,page,pageSize}`. Default sort is `-createdAt`. Runtime-supported sort keys are only `createdAt`, `name`, `fullName`, `date`, `status`, `code`, `tokenNumber`, `sortOrder`, and `email`; a leading `-` reverses ordering. Sorting compares stringified values with numeric collation. [source: `lib/api-spec/openapi.yaml:432-444`; `artifacts/api-server/src/lib/store.ts:47-52`]
- Shared filters are exact matches for `clinicId`, `branchId`, `doctorId`, `patientId`, `status`, `role`, `category`, `parentId`, `gender`, `city`, `specializationId`, `source`, `entityType`, `actorId`, and `date`. For enriched users/doctors, clinic/branch filters also match membership arrays. `from` / `to` compare the record's `date`, or otherwise the UTC date portion of `createdAt`. Search is case-insensitive over `name`, `fullName`, `email`, `mobile`, `code`, `reference`, `patientName`, `doctorName`, and `summary`. Filters are applied only after authorization and cannot widen scope. [source: `artifacts/api-server/src/lib/store.ts:37-45`]

2. Authentication, security, and identity

Clerk identity

Clerk middleware validates the authenticated identity. `requireIdentity` requires a Clerk `userId` (401 otherwise). `requireUser` additionally requires a linked ClinicFlow user and `status=active` (403 otherwise). On lookup, an unlinked Clerk identity can be attached to a pre-created user only when a verified Clerk email exactly matches the stored lower-case email and that user has no `clerkId`. [source: `artifacts/api-server/src/lib/auth.ts:7-30`]

The production-only `/api/__clerk` proxy targets Clerk's frontend API. It is inactive outside production and also becomes a no-op if the server-side Clerk key is absent. It derives the public host from the first `x-forwarded-host` value, sets Clerk proxy headers, forwards the first client IP, strips hop-by-hop

response headers, and buffers length-unknown responses so a `Content-Length` can be emitted. [source: `artifacts/api-server/src/middlewares/clerkProxyMiddleware.ts:26-28,46-64,66-145`]

This backend never accepts a password and does not implement password storage. The “password reset” operation only tells the caller to use Clerk’s sign-in recovery; it does not send an email itself. [source: `artifacts/api-server/src/routes/resources.ts:200-205`]

Transport/request controls

- Helmet is enabled, but its content-security-policy middleware is disabled. [source: `artifacts/api-server/src/app.ts:35`]
- CORS is configured with credentials enabled and `origin:false` , so the API itself emits no permissive CORS origin. [source: `artifacts/api-server/src/app.ts:37`]
- `trust proxy` is set to one hop. [source: `artifacts/api-server/src/app.ts:34`]
- Every `/api` request is limited to 180 requests per 60 seconds by `express-rate-limit` ; standard draft-8 headers are emitted and legacy headers are disabled. [source: `artifacts/api-server/src/app.ts:49`]
- A non-GET/HEAD/OPTIONS request is rejected when `Sec-Fetch-Site` is `cross-site` , or when `Origin` 's host differs from forwarded host/host. If no Origin is supplied, an `Authorization` header beginning exactly `Bearer` is required. This is an origin/header check, not a CSRF token mechanism. [source: `artifacts/api-server/src/app.ts:50-61`]
- Request logging records request ID, method, URL without query string, and response status. OTP delivery explicitly avoids logging SMS payloads. [source: `artifacts/api-server/src/app.ts:15-33` ; `artifacts/api-server/src/lib/otp-delivery.ts:16-17`]

Onboarding and profile linkage

- `GET /me` requires Clerk identity but not onboarding. It returns `{clerkId,user,doctorId,patientId,needsOnboarding}` ; `user` is null before onboarding. An existing inactive account is rejected. [source: `artifacts/api-server/src/routes/identity.ts:10-14`]
- `POST /onboarding` requires a verified primary Clerk email and no existing profile. Body: required `fullName` (non-empty); optional `intent` (`patient` default or `doctor`), `mobile` , and `termsAccepted` . `termsAccepted` is accepted but not enforced here. A transaction takes a Clerk-ID advisory lock, creates the user, then a doctor profile for doctor intent or patient profile otherwise, and writes an audit row. Patient is the fallback role. [source: `lib/api-spec/openapi.yaml:479-486` ; `artifacts/api-server/src/routes/identity.ts:15-30`]
- `PATCH /me` body fields are optional `fullName` (non-empty), `mobile` , and `photoUrl` . It updates the user; mirrors name into a patient profile and resets mobile verification unless the number is unchanged; or merges the body into a doctor profile. The audit record is written against the pre-update user object. [source: `lib/api-spec/openapi.yaml:473-478` ; `artifacts/api-server/src/routes/identity.ts:32-46`]

OTP/mobile verification

OTP verifies a mobile number for an already authenticated, active ClinicFlow user; it does not authenticate the user.

- `POST /otp/request` : body `{mobile}` where mobile must match `^\+[1-9][0-9]{7,14}$` . The endpoint has the global limit plus an OTP-router limit of 30 requests per 15 minutes. Per user, an advisory lock enforces a 60-second resend cooldown and at most five challenges created in the preceding hour. Expiry comes from platform settings but is clamped to 60–900 seconds (default 300). A cryptographically generated six-digit code is stored only as HMAC-SHA256 of `challengeId:code` , keyed by the required `SESSION_SECRET` . Delivery occurs before the new row is inserted; all previous challenges for the user are then marked consumed. Response is `{challengeId,expiresAt,resendAfterSeconds:60,provider}` and `Cache-Control:no-store` . [source: `artifacts/api-server/src/routes/otp.ts:11-59` ; `lib/api-spec/openapi.yaml:517-530`]
- `POST /otp/verify` : body requires `challengeId` and a 4–8 digit `code` . It locks per user and row-locks the matching challenge belonging to that user. Missing, consumed, or expired challenges fail; attempt limit comes from settings and is clamped to 1–10 (default 5). Every submitted code increments attempts before constant-time digest comparison, including the successful attempt. On success the challenge is consumed, the linked patient's mobile is set and verified, and the user's mobile is updated in one transaction. Response is `{verified:true,mobile,verifiedAt}` with `Cache-Control:no-store` . Failed attempt counts are deliberately returned from, rather than thrown within, the transaction so they commit. [source: `artifacts/api-server/src/routes/otp.ts:62-89` ; `lib/api-spec/openapi.yaml:531-543`]
- Delivery is either an explicitly enabled development provider (`NODE_ENV=development` and `OTP_PROVIDER=development`) or Twilio through Replit Connectors when provider identifiers have the expected shape. Development mode returns `developmentCode` ; production never does. Missing configuration and delivery rejection return coded 503 errors. The SMS includes the code, rounded-up expiry in minutes, and a do-not-share warning. [source: `artifacts/api-server/src/lib/otp-delivery.ts:4-37` ; `artifacts/api-server/src/routes/otp.ts:54-59`]
- There is no challenge cleanup job in this source; consumed/expired rows remain stored. OTP request rate limiting is account-aware only for the database rules and IP-derived for `express-rate-limit` . Mobile numbers are not unique, and a request may target a number different from the user's current number. [source: `lib/db/src/schema/core.ts:71-75` ; `artifacts/api-server/src/routes/otp.ts:20-53`]

3. Authorization and tenancy

Scope model

`assignments` links a user to a clinic, optionally to a branch. A user's `clinicIds` is every distinct clinic in those rows; `branchIds` contains only non-null branch links. `superAdmin` bypasses scope. For everyone

else a clinic must be in `clinicIds` . A branch restriction is enforced by the generic `scope` helper only for receptionists; clinic admins and doctors pass branch scope when the clinic is assigned. [source: `artifacts/api-server/src/lib/auth.ts:23-27,33-37` ; `lib/db/src/schema/core.ts:18-21`]

Important observed tenancy boundaries:

- There is one shared database and shared platform settings singleton, not a database/schema/settings record per tenant.
- Masters are globally readable to every active authenticated user and globally writable only by super admins. [source: `artifacts/api-server/src/lib/auth.ts:38-40` ; `artifacts/api-server/src/routes/resources.ts:36`]
- Clinic visibility is ownership or assignment. Branches and most clinic-owned resources use assignment scope. Doctors are visible to themselves, or clinic admins/receptionists with intersecting assignments; receptionists additionally need an intersecting doctor branch.
- A doctor can read only own appointments, schedules, availability exceptions, and QR rows; patient records are visible to a doctor only through an appointment with that doctor. A doctor's generic clinic/branch reads can nevertheless follow assigned clinic scope, and a doctor may create clinics and branches they own as described below. [source: `artifacts/api-server/src/lib/auth.ts:41-52` ; `artifacts/api-server/src/routes/resources.ts:41-55`]
- A patient can read only their own patient row and own appointments. Masters are the exception because all authenticated users can read them. Queue access is a separate relationship check. [source: `artifacts/api-server/src/lib/auth.ts:39,42-49` ; `artifacts/api-server/src/routes/queue.ts:15-26`]
- Patient registration has a single nullable `clinicId` / `branchId` ; it is not a many-to-many tenancy model. Staff visibility may also arise from appointment relationship. Non-super-admin staff cannot move an existing patient registration to another clinic/branch. [source: `lib/db/src/schema/core.ts:29-33` ; `artifacts/api-server/src/routes/resources.ts:46-51`]
- Assignments are replaced wholesale on user/doctor save. Each clinic ID creates a clinic-level row and each branch ID creates another row carrying its clinic, so duplicates at clinic level are possible by design and no database uniqueness constraint prevents duplicate assignments. A branch must belong to one of the submitted clinic IDs. [source: `artifacts/api-server/src/lib/auth.ts:58-69` ; `lib/db/src/schema/core.ts:18-21`]

Role capability matrix

“Scoped” below always includes the row-level rules above and active-account requirement.

Capability	superAdmin	clinicAdmin	doctor	receptionist	patient
Read public endpoints	yes	yes	yes	yes	yes
Read masters	yes	yes	yes	yes	yes

Capability	superAdmin	clinicAdmin	doctor	receptionist	patient
Mutate masters/settings	yes	no	no	no	no
List users	all	scoped, excludes super admins	no	no	no
Create/update/deactivate users	all; last active super admin protected	scoped; can administer doctor/receptionist /patient only; cannot change existing role	no	no	no
Read clinics/branches	all	scoped	owned/assigned scope	scoped, branch-restricted	no
Create clinic	yes	no	yes, becomes owner and assigned	no	no
Update/deactivate clinic	yes	scoped	owned only	no	no
Create/update/deactivate branch	yes	scoped	owned clinic only	no	no
Read doctors	all	scoped	self	scoped	no
Create doctor	yes	scoped admin	no	no	no
Update doctor	yes	scoped admin; rejected if doctor also assigned outside admin's clinics	self; cannot add assignments	no	no
Deactivate doctor	yes	scoped admin	self is technically authorized by the same write rule	no	no
Read patients	all	registration/appointment scope	appointment relationship	registration/appointment scope	self
Create patient	yes	scoped	no	scoped	no
Update patient	yes	registering scope	no	registering scope	self demographics only; no registration/status fields
Deactivate patient	yes	scoped	no	scoped	explicitly forbidden
Mutate schedules/exceptions	yes	scoped	own doctor ID	no	no
Mutate QRs	yes	scoped	own doctor ID when supplied/own-readable row	scoped receptionist	no

Capability	superAdmin	clinicAdmin	doctor	receptionist	patient
Book	any valid scope	valid scope	own doctor only	valid branch scope	self only, source online or qr
Appointment queue actions	all scoped	scoped	own	scoped	only cancel own
Read reports	yes	scoped	own appointment scope	scoped	no
Read audit logs	all	scoped clinic logs	no	no	no
Dashboard	scoped	scoped	scoped	scoped	own appointments and own patient count

[source: `artifacts/api-server/src/lib/auth.ts:32-56` ; `artifacts/api-server/src/routes/resources.ts:32-81,166-205` ; `artifacts/api-server/src/routes/appointments.ts:21-24` ; `artifacts/api-server/src/lib/appointments.ts:26-35` ; `artifacts/api-server/src/routes/reporting.ts:16-48`]

4. Endpoint reference

All endpoints except those marked public require Clerk bearer/session authentication; most further require an active ClinicFlow user.

Health and public discovery

Method/path	Authentication	Inputs and behavior
<code>GET /healthz</code>	public	Returns <code>{status:"ok"}</code> . It does not query the database or integrations.
<code>GET /public/clinics</code>	public	Active clinics only. Query: <code>search</code> , <code>page</code> , <code>pageSize</code> . Removes <code>ownerId</code> but otherwise returns flattened clinic fields.
<code>GET /public/branches</code>	public	Active branches whose clinic is active. Query: <code>clinicId</code> , <code>page</code> , <code>pageSize</code> .

Method/path	Authentication	Inputs and behavior
GET /public/doctors	public	Active doctor/account profiles with at least one active assigned branch; assignment arrays are reduced to active clinics/branches. Query: search , clinicId , branchId , specializationId , page , pageSize . Returns only id , fullName , photoUrl , specializationName , qualificationNames , about , experienceYears , consultationFee , clinicIds , branchIds .
GET /public/availability	public	Required query doctorId , branchId , date ; returns computed availability described in §6.
GET /public/qr/:reference	public	Resolves an active QR plus active clinic, optional branch, and optional doctor/account. Returns identifiers/names only; invalid, revoked, or inactive context is 404.

[source: artifacts/api-server/src/routes/health.ts:6-9 ; artifacts/api-server/src/routes/public.ts:10-45]

Identity and OTP

GET /me , PATCH /me , POST /onboarding , POST /otp/request , and POST /otp/verify are specified in §2. GET /me and onboarding require only Clerk identity at entry; profile update and OTP require an active application user.

Generic resources

The same runtime loop implements:

- GET /{resource} — scoped/filterable paginated list.
- GET /{resource}/:id — scoped item.
- POST /{resource} — create, HTTP 201.
- PATCH /{resource}/:id — validate using the full create schema, merge with old row, update.
- DELETE /{resource}/:id — soft-deactivate by setting relational status="inactive" , HTTP 204.

There is no hard-delete endpoint.

Resources are clinics , branches , doctors , users , patients , masters , schedules , availability-exceptions , and qrs . Thus the implementation includes GET /schedules/:id and GET /availability-exceptions/:id , even though those two item reads are omitted from the current OpenAPI paths.

[source: artifacts/api-server/src/routes/resources.ts:14-24,166-199 ; compare lib/api-spec/openapi.yaml:264-307]

Declared list query fields (in addition to the shared pagination behavior) are:

Resource	Query fields
clinics	search , status , city , page , pageSize , sort
branches	clinicId , search , status , page , pageSize , sort
doctors	search , clinicId , branchId , status , specializationId , page , pageSize , sort
users	search , role , clinicId , branchId , status , page , pageSize , sort
patients	search , clinicId , branchId , gender , page , pageSize , sort
masters	category , parentId , search , status , page , pageSize , sort
schedules	doctorId , clinicId , branchId , page , pageSize
availability exceptions	doctorId , branchId , from , to , page , pageSize
QRs	clinicId , branchId , doctorId , status , page , pageSize

[source: `lib/api-spec/openapi.yaml`:96-99,122-125,148-151,176-179,210-214,237-241,264-267,286-289,361-364]

Resource fields and validation

Fields not described as required are optional in the generated request schema. Because create schemas are reused, “required” applies to PATCH too.

Resource	Required request fields	Optional request fields and constraints	Server behavior
clinics	name non-empty, address	country , state , city , area , pincode , phone , email-format email , description , clinicTypeId , categoryId , status active/inactive	Master references must be active categories <code>clinicType / clinicCategory</code> . Owner is creator and cannot be transferred. Code generated if absent.
branches	clinicId , name non-empty, address	city , state , pincode , phone , timezone default Asia/Kolkata , status	Timezone is validated by <code>Intl</code> ; branch clinic/scope is checked; code generated.

Resource	Required request fields	Optional request fields and constraints	Server behavior
doctors	fullName non-empty, email-format email	mobile , photoUrl , gender , real dateOfBirth , specializationId , qualificationIds[] , registrationNumber , integer experienceYears>=0 , about , consultationFee>=0 , languages[] , clinicIds[] , branchIds[] , status	Email lower-cased. Specialization and qualifications must be active correct-category masters. Creates/updates linked user role doctor. Assignments replaced wholesale.
users	fullName non-empty, email-format email , role	mobile , status , clinicIds[] , branchIds[]	Email lower-cased. New doctor/patient roles create typed profiles. Existing role cannot change. Linked doctor's status follows a supplied user status; linked patient's changed mobile resets verification.
patients	fullName non-empty, mobile	email-format email , real dateOfBirth , integer age 0-130, gender , address , emergency contact fields, clinicId , branchId , status	Mobile is additionally international-format validated and is mandatory in save. Code generated. Changing it resets mobileVerified . Patient self-update cannot submit clinic/branch/status.
masters	category , name non-empty, code non-empty	nullable parentId , integer sortOrder , status	Category enum: country , state , city , area , pincode , clinicType , clinicCategory , clinicStatus , specialization , qualification , department , consultationType , appointmentStatus , appointmentType , bookingSource , cancellationReason , queueStatus , tokenPrefix , queueType , queuePriority , userRole , userStatus . Parent must exist and cannot be self.

Resource	Required request fields	Optional request fields and constraints	Server behavior
schedules	<code>doctorId</code> , <code>clinicId</code> , <code>branchId</code> , integer <code>dayOfWeek</code> 0-6 (Sunday 0), <code>isOpen</code> , <code>startTime</code> , <code>endTime</code> , <code>tokenPrefix</code> 1-8 chars, integer <code>maxTokens</code> >=1 , integer <code>consultationMinutes</code> >=1	nullable <code>breakStart</code> / <code>breakEnd</code> ; <code>timezone</code> default Asia/Kolkata ; integer <code>bufferMinutes</code> >=0 default 0; <code>queueMode</code> default <code>mixed</code> (<code>mixed</code> , <code>appointmentsOnly</code> , <code>walkInsOnly</code>); <code>queueOpenTime</code> , <code>queueCloseTime</code>	Doctor must be assigned/active at branch. Time/session and collision rules are in §6. Although OpenAPI regexes only <code>startTime</code> , runtime validates all used times as strict 24-hour HH:mm .
availability-exceptions	<code>doctorId</code> , <code>branchId</code> , real date , <code>isClosed</code> , reason	nullable <code>startTime</code> , <code>endTime</code> , <code>breakStart</code> , <code>breakEnd</code> , nullable integer <code>maxTokens</code> >=1	Unique per doctor/branch/date. Open overrides require a base weekly schedule and are time/cross-branch overlap checked; closed overrides skip time validation.
qrs	<code>name</code> , <code>clinicId</code>	nullable <code>branchId</code> , nullable <code>doctorId</code> , <code>status</code>	Creates a random immutable- until-regenerated <code>publicReference</code> and <code>bookingUrl</code> = /book/{reference } . Context consistency/scope is checked when IDs are truthy.

[source: `lib/api-spec/openapi.yaml:544-735,851-869` ; `artifacts/api-server/src/routes/resources.ts:32-164` ; `artifacts/api-server/src/lib/availability.ts:4-31`]

Governed master categories accept only these codes: `userRole` = the five application roles; `appointmentStatus` and `queueStatus` = the eight appointment statuses; `bookingSource` = `online` , `walkIn` , `phone` , `qr` ; `queueType` = `mixed` , `appointmentsOnly` , `walkInsOnly` ; `userStatus` and `clinicStatus` = `active` , `inactive` . Existing governed category/code cannot be changed. Other master categories are not code-governed beyond category enum and uniqueness. [source: `artifacts/api-server/src/routes/resources.ts:25-31,76-80`]

Creating a doctor or user first looks for an existing DB email and refuses duplicates. It then links an existing Clerk user only if that Clerk account has that verified email and no DB profile; otherwise it asks Clerk to create an invitation. Clerk invitation/link lookup is external to the subsequent PostgreSQL transaction, so invitation creation and DB creation are not atomic. [source: `artifacts/api-server/src/routes/resources.ts:82-100`]

Additional endpoints:

- `POST /users/:id/password-reset` : super admin or scoped clinic admin; returns HTTP 202 explaining that the user must use `/sign-in` → Forgot password. It writes audit action `recoveryInstructions` , but sends nothing. [source: `artifacts/api-server/src/routes/resources.ts:200-205`]

- `POST /qrs/:id/regenerate` : authorized QR writer; replaces the public reference and booking URL transactionally and audits `regenerate` . The old reference immediately ceases to resolve because only the replacement is stored. [source: `artifacts/api-server/src/routes/resources.ts:206-214`]

Appointments

- `GET /appointments` : active application user; scoped list. Query fields: `search` , `clinicId` , `branchId` , `doctorId` , `patientId` , `date` , `from` , `to` , appointment `status` , booking `source` , `page` , `pageSize` , `sort` .
- `GET /appointments/:id` : scoped item.
- `POST /appointments` : body requires `patientId` , `doctorId` , `clinicId` , `branchId` , real `date` , and source (`online` , `walkIn` , `phone` , `qr`). Optional: `appointmentTypeId` , `consultationTypeId` , `qrReference` , `requestId` , `termsAccepted` , and `notes` up to 1000 characters.
- `POST /appointments/:id/actions` : body requires action; optional `reason` , `cancellationReasonId` , `expectedStatus` . `cancellationReasonId` is accepted by validation but not looked up or used by transition logic. `reason` , when supplied, appears in the JSON history event but not the relational history row.

[source: `lib/api-spec/openapi.yaml:308-341,755-811` ; `artifacts/api-server/src/routes/appointments.ts:12-72` ; `artifacts/api-server/src/lib/appointments.ts:26-53`]

Booking rules:

1. Patients may book only their linked patient ID and only source `online` or `qr` ; staff must pass scope, and doctors may book only themselves as doctor.
2. A transaction takes advisory locks for the doctor and doctor/branch/date queue.
3. `requestId` , when supplied, is idempotent per actor. If an original exists with the same patient/doctor/branch/date, it is returned; clinic/source and all other fields are not compared. A mismatched core tuple returns 409. Database uniqueness is `(actorId,requestId)` ; PostgreSQL permits multiple null request IDs.
4. The same patient cannot have another non-terminal (`cancelled` , `completed` , `noShow` are terminal for this rule) appointment for the doctor/branch/date.
5. Patient, doctor account/profile, clinic, and branch must be active; doctor must have a branch assignment. Patient mobile must be valid international format. Staff must be able to read the patient.
6. Computed availability must be available and its branch-derived clinic must equal submitted `clinicId` .
7. Queue mode rejects walk-ins for `appointmentsOnly` , and rejects every non-walk-in source for `walkInsOnly` .
8. Walk-ins must be for the branch-local current date. Same-day booking must be before queue close. Walk-ins must additionally be after queue open (or session start) and outside the configured

break. Online/phone/QR same-day bookings do not enforce queue-open time or break.

9. Platform mobile-verification policy is enforced. Patient-originated bookings require `truthy termsAccepted` ; staff bookings do not.
10. QR source requires `qrReference` ; the active QR's clinic must match and any QR branch/doctor constraints must match.
11. Appointment/consultation type IDs, if supplied, must be active masters of the correct category.
12. Token number is one greater than the maximum token ever allocated for that doctor/branch/date, including terminal appointments; cancelled tokens are not reused.

[source: `artifacts/api-server/src/routes/appointments.ts:21-65` ; `lib/db/src/schema/core.ts:42-56`]

The stored appointment snapshots patient/doctor/clinic/branch names, patient code, timezone and session times, plus a generated reference/token and JSON status history. Later edits to those entities do not update the snapshot. `actorId` and `requestId` are deliberately removed from appointment API views. [source: `artifacts/api-server/src/routes/appointments.ts:61-64` ; `artifacts/api-server/src/lib/appointments.ts:21-24`]

Queue

- `GET /queue` : required `doctorId` , `branchId` , `date` ; optional `appointmentId` . All callers need an active doctor/account/clinic/branch assignment context. Non-patients need scope and a doctor caller must be that doctor. A patient must have an appointment in that queue; if `appointmentId` is supplied, it must identify their matching appointment. Patients receive aggregates and `ownEntry` , but no `entries` ; staff receive all queue appointment views.
- `POST /queue/call-next` : required body `doctorId` , `branchId` , `date` ; super admin, clinic admin, doctor, or receptionist with queue scope. It locks the queue, rejects if any appointment is called/in consultation, then calls the first waiting appointment or returns `{appointment:null}` .

Waiting order is `waitingAt` falling back to `createdAt` , then token number. `currentToken` is the first row found in called/in-consultation state; `nextToken` is the first waiting token. Counts include all appointments for the queue date and `total` includes terminal/cancelled entries. A patient's `patientsAhead` includes waiting predecessors and one current called/in-consultation patient; booked/checked-in patients are placed behind all waiting entries; terminal/current own states report zero. Estimated wait is `patientsAhead * (weekly schedule consultationMinutes default 10 + bufferMinutes default 0)` and does not apply date exceptions or actual progress. Poll interval is hard-coded to 30 seconds. [source: `artifacts/api-server/src/routes/queue.ts:10-38`]

Dashboard, reports, audit, and settings

- `GET /dashboard` : optional `date` , `clinicId` , `branchId` , `doctorId` . Date defaults to “today” in platform timezone. Appointment metrics are scoped then filtered. Active entity counts include only active rows and are separately scoped/filterable by clinic/branch. `activeQueues` counts distinct doctor/branch pairs with waiting/called/in-consultation rows. `currentToken` is the first active row,

not explicitly sorted. Recent appointments are eight newest by `createdAt` . Recent activity is only included for super/clinic admins and is eight newest audit rows in scope. [source: `artifacts/api-server/src/routes/reporting.ts:11-25`]

- `GET /reports` : super admin, clinic admin, doctor, or receptionist. Optional `from` , `to` , `clinicId` , `branchId` , `doctorId` , `groupBy` (`date` default, `clinic` , `doctor`). Range defaults from first day of the platform-timezone current month through today and must be ordered. Metrics count appointments by current status and average only rows with numeric measured durations. Registrations are patient rows by `createdAt` . Date and clinic groups may be created from registrations without appointments; doctor groups are not. Doctor-group registrations count scoped registrations whose patient appears in one of that doctor's grouped appointments; clinic groups use patient registration clinic. Labels use the first appointment's snapshot name or key. [source: `artifacts/api-server/src/routes/reporting.ts:27-37`]
- `GET /audit-logs` : super admin or clinic admin. Query `search` , `entityType` , `actorId` , `clinicId` , `from` , `to` , `page` , `pageSize` . It enriches current actor name/role. Scope uses audit `clinicId` ; super admin sees all, while clinic admin sees only assigned non-null clinic IDs. [source: `artifacts/api-server/src/routes/reporting.ts:39-44` ; `artifacts/api-server/src/lib/auth.ts:33-37`]
- `GET /settings` : any active application user. Returns defaults merged with persisted singleton `settings.id="platform"` , plus live `otpProviderConfigured` and forced `queuePollSeconds:30` .
- `PATCH /settings` : super admin only. Optional fields: `platformName` ; email-format `supportEmail` ; `supportPhone` ; valid IANA `timezone` ; integer `bookingHorizonDays>=1` ; integer `cancellationCutoffMinutes>=0` ; `requireMobileVerification` ; integer `otpExpirySeconds` 60-900; integer `otpMaxAttempts` 1-10; integer `sessionTimeoutMinutes>=5` ; `notificationsEnabled` ; `logoUrl` ; `primaryColor` ; `termsUrl` ; `privacyUrl` . It merges and upserts the singleton and audits the update. `otpProviderConfigured` cannot be persisted.

Defaults are `platformName=ClinicFlow` , `timezone=Asia/Kolkata` , `bookingHorizonDays=60` , `cancellationCutoffMinutes=0` , `requireMobileVerification=false` , `otpExpirySeconds=300` , `otpMaxAttempts=5` , `sessionTimeoutMinutes=60` , `notificationsEnabled=false` , and `queuePollSeconds=30` . `sessionTimeoutMinutes` does not configure Clerk sessions in this source. `notificationsEnabled` is stored but no general notification dispatcher exists; implemented outbound messages are OTP SMS and Clerk invitations. [source: `artifacts/api-server/src/lib/store.ts:28-35` ; `artifacts/api-server/src/routes/reporting.ts:45-54` ; `lib/api-spec/openapi.yaml:937-962`]

5. Appointment state machine

Action	Allowed from	Result	Timestamp/data effect
<code>checkIn</code>	<code>booked</code>	<code>checkedIn</code>	<code>checkedInAt</code>
<code>enqueue</code>	<code>checkedIn</code>	<code>waiting</code>	<code>waitingAt</code>
<code>call</code>	<code>waiting</code>	<code>called</code>	<code>calledAt</code>

Action	Allowed from	Result	Timestamp/data effect
start	called	inConsultation	consultationStartedAt ; computes waitMinutes from waitingAt
complete	inConsultation	completed	completedAt ; computes consultationMinutesActual from consultation start
noShow	waiting , called	noShow	no dedicated timestamp
requeue	noShow	waiting	replaces/sets waitingAt
cancel	booked , checkedIn , waiting	cancelled	cancelledAt

Every transition:

- verifies row scope, takes queue advisory locks (unless the caller already holds them), reloads the row, validates the transition, and optionally enforces `expectedStatus` ;
- permits patients only `cancel` on their own appointment; all other actions require a scoped staff role;
- revalidates active doctor/clinic/branch context for all actions except cancel, complete, and no-show;
- permits non-cancel queue actions only on the appointment's local current date;
- refuses call/start if a different row is called or in consultation;
- appends a JSON history event, inserts a relational `appointment_history` row, and inserts an audit summary in one transaction.

Cancellation computes minutes until appointment `startTime` using the appointment's stored timezone/date and requires that value to be at least `cancellationCutoffMinutes` . With the default cutoff 0, cancellation is accepted until the stored start minute. There is no transition out of completed/cancelled, and no direct skip between states. `allowedActions` in responses is derived only from current state and, for patients, reduced to cancel; for non-patients it does not pre-evaluate scope, date, cutoff, active-context, or one-current-patient rules, so an advertised action can still fail. [source: `artifacts/api-server/src/lib/appointments.ts:7-53`]

6. Availability, timezones, capacity, and concurrency

Weekly schedules and exceptions

- Time strings used by runtime must be exactly `HH:mm` in 24-hour range. Open sessions require start before end; overnight sessions are unsupported. Break start/end must be both present or both absent, ordered, and within the session. Queue open must precede session end. Queue close

must be at/before session end and after queue open or session start. Closed schedule/exception validation returns early. [source: `artifacts/api-server/src/lib/availability.ts:4-24`]

- A schedule collision check rejects another active schedule for the same doctor/day if it is the same branch regardless of open/closed state, or, when both are open, if time ranges overlap. Differing timezones are conservatively treated as collision rather than converted. There is no database unique constraint for schedules; this rule is transaction/advisory-lock enforced by application code. [source: `artifacts/api-server/src/routes/resources.ts:99-110` ; `lib/db/src/schema/core.ts:34-37`]
- A non-closed date exception requires an active weekly base schedule at that branch/day. It overlays non-null submitted values, while nullable break fields may explicitly clear breaks, validates effective times, and rejects overlap with another branch's active open schedule. Closed exceptions do not require a base schedule in the save path. The database permits exactly one exception per doctor/branch/date. [source: `artifacts/api-server/src/routes/resources.ts:111-119` ; `lib/db/src/schema/core.ts:38-41`]

Computation

Availability validates the date, verifies active doctor profile, linked account, clinic, branch, and exact doctor-branch assignment, and selects the first active weekly schedule matching UTC weekday. It selects the first active matching exception. Exception overrides only `startTime` , `endTime` , `breakStart` , `breakEnd` , and `maxTokens` ; it does not override timezone, token prefix, consultation/buffer minutes, queue mode, or queue open/close. Effective timezone is schedule timezone, else branch timezone, else `Asia/Kolkata` . [source: `artifacts/api-server/src/lib/availability.ts:25-42`]

Booked capacity counts every appointment on doctor/branch/date except `cancelled` ; completed and no-show appointments therefore still consume capacity. Availability is false, with later checks able to replace earlier reasons, when there is no open weekly schedule, a closed exception, the session date/end is in the past, the date exceeds the platform booking horizon, or capacity is exhausted. No explicit lower-bound horizon check is needed because past dates are separately rejected. `maxTokens` defaults to 0, consultation minutes to 10, token prefix to `A` , queue mode to `mixed` , and buffer to 0 when values are missing/falsy. [source: `artifacts/api-server/src/lib/availability.ts:33-51`]

The runtime availability object contains `doctorId` , `branchId` , branch-derived `clinicId` , `date` , `available` , nullable `reason` , nullable start/end/break times, `timezone` , `maxTokens` , `bookedTokens` , `remainingTokens` , `consultationMinutes` , `tokenPrefix` , `queueMode` , optional queue open/close, and `bufferMinutes` . The current OpenAPI `Availability` schema does not declare the final four queue-related runtime properties. [source: `artifacts/api-server/src/lib/availability.ts:51` ; `lib/api-spec/openapi.yaml:735-754`]

Dates and clock times are stored separately as text. "Now" is converted through `Intl.DateTimeFormat` in the effective IANA timezone. Weekday selection uses noon UTC for the supplied calendar date. Appointment transition timing durations use absolute UTC instants (`Date.now()`) against stored ISO timestamps, while session/cutoff checks use local calendar date plus minute-of-day. [source:

```
artifacts/api-server/src/lib/availability.ts:8-13,33-36 ; artifacts/api-server/src/lib/appointments.ts:36-48 ]
```

Locking and database enforcement

Booking and queue changes take PostgreSQL transaction advisory locks on `schedules:{doctorId}` and `{doctorId}:{branchId}:{date}`. Schedule saves take a resource-derived doctor lock; exception saves additionally take the schedule doctor lock. Queue rows are re-read after locking. [source: `artifacts/api-server/src/lib/appointments.ts:17-20,26-31` ; `artifacts/api-server/src/routes/resources.ts:99-102`]

Database constraints provide a second layer:

- unique token number per doctor/branch/date;
- positive token number;
- unique `(actorId,requestId)` ;
- unique non-terminal appointment per patient/doctor/branch/date;
- at most one called or in-consultation appointment per doctor/branch/date;
- constrained appointment status values.

The idempotency lookup and insertion occur under the queue lock, and the unique actor/request index protects reuse across different queues as well. There is no generic optimistic version column; `expectedStatus` is the transition-specific stale-write guard. [source: `lib/db/src/schema/core.ts:47-56` ; `artifacts/api-server/src/routes/appointments.ts:25-32` ; `artifacts/api-server/src/lib/appointments.ts:29-32`]

7. Audit and reporting semantics

Audit rows are append-only by API convention: there are no audit mutation routes. Each contains actor, derived clinic scope, action, entity type/id, fixed summary text `${action} ${type} record`, and creation time. The helper derives clinic from `row.clinicId`, or from the clinic's own ID only when entity type is `clinics`. Consequently platform/user/master/settings audits generally have null clinic scope; clinic admins cannot see null-scoped logs. No old/new values, request ID, IP, reason, or payload are stored. The database foreign keys do not use cascade deletion. [source: `artifacts/api-server/src/lib/store.ts:25-27` ; `lib/db/src/schema/core.ts:66-70`]

Audited operations are onboarding, self profile update, generic create/update/deactivate, appointment booking and every appointment transition, password-recovery instructions, QR regeneration, and settings update. OTP request/verify, reads, and failed writes are not audited. Generic deactivate audits the pre-deactivation object; self-profile update similarly passes the pre-update user; the summary does not distinguish those details. [source: `artifacts/api-server/src/routes/identity.ts:21-27,32-45` ; `artifacts/api-server/src/routes/resources.ts:149-162,184-213` ; `artifacts/api-`

```
server/src/routes/appointments.ts:63-65 ; artifacts/api-server/src/lib/appointments.ts:50-52 ;  
artifacts/api-server/src/routes/reporting.ts:46-53 ]
```

Measured wait is recorded only at `start`, from the latest `waitingAt` (requeue resets it). Measured consultation duration is recorded only at `complete`. Reports average only rows with numeric values, return zero for no samples, and do not round. Status metrics are current-state counts rather than event counts. [source: `artifacts/api-server/src/lib/appointments.ts:46-50` ; `artifacts/api-server/src/routes/reporting.ts:11-14`]

8. Database schema and migrations

All primary keys are text. Unless stated otherwise, foreign keys use PostgreSQL `NO ACTION` for update/delete. JSONB `data` columns are non-null and default to `{}`. `created_at` columns shown are timezone-aware, non-null, and default to `now()`. [source: `lib/db/src/schema/core.ts:4-75` ; `lib/db/drizzle/0000_clinicflow_initial.sql:1-199`]

users

- `id` PK; nullable unique `clerk_id` ; unique non-null `email` ; non-null `full_name` ; nullable `mobile` ; non-null `role` ; non-null `status` default `active` ; `data` ; `created_at` .
- Check `role` IN (`superAdmin`, `clinicAdmin`, `doctor`, `receptionist`, `patient`) . There is no database check on status or mobile format.
- Referenced by clinic ownership, assignments, doctor/patient profiles, appointment actors/history, audit actors, and OTP challenges. [source: `lib/db/src/schema/core.ts:7-11`]

clinics

- `id` PK; nullable `owner_id` -> `users.id` ; non-null `status` default `active` ; `data` ; `created_at` .
- Index on `owner_id` . Referenced by branches, assignments, patients, schedules, appointments, QRs, and audit logs. [source: `lib/db/src/schema/core.ts:12-14`]

branches

- `id` PK; non-null `clinic_id` -> `clinics.id` ; non-null `status` default `active` ; `data` ; `created_at` .
- Index on `clinic_id` . Referenced by assignments, patients, schedules, exceptions, appointments, and QRs.
- The database does not enforce that a row elsewhere containing both clinic and branch uses the branch's clinic; application writes check this in several paths. [source: `lib/db/src/schema/core.ts:15-17`]

assignments

- `id` PK; non-null `user_id -> users.id` ; non-null `clinic_id -> clinics.id` ; nullable `branch_id -> branches.id` .
- Indexes on `user_id` and `(clinic_id,branch_id)` .
- No created time, status, unique constraint, or database-level branch/clinic consistency constraint. [source: `lib/db/src/schema/core.ts:18-21`]

masters

- `id` PK; non-null `category` ; non-null `code` ; nullable self-reference `parent_id -> masters.id` ; non-null `status` default `active` ; `data` .
- Unique index `(category,code)` . No created time. The self-reference was added by migration `0001` . [source: `lib/db/src/schema/core.ts:22-24` ; `lib/db/drizzle/0001_clinicflow_integrity.sql:1`]

doctors

- `id` PK; non-null unique `user_id -> users.id` ; nullable `specialization_id -> masters.id` ; non-null `status` default `active` ; `data` .
- No created time. Clinic/branch membership is through the linked user's assignments, not doctor columns. [source: `lib/db/src/schema/core.ts:25-28`]

patients

- `id` PK; nullable unique `user_id -> users.id` ; nullable `clinic_id -> clinics.id` ; nullable `branch_id -> branches.id` ; non-null `mobile` default empty string; non-null `mobile_verified` default false; non-null `status` default `active` ; `data` ; `created_at` .
- Indexes on `(clinic_id,branch_id)` and `mobile` ; `mobile` is not unique. Standalone staff-created patients may have no user identity. The `mobile` index was added in migration `0001` . [source: `lib/db/src/schema/core.ts:29-33` ; `lib/db/drizzle/0001_clinicflow_integrity.sql:3`]

schedules

- `id` PK; non-null `doctor_id -> doctors.id` ; non-null `clinic_id -> clinics.id` ; non-null `branch_id -> branches.id` ; non-null integer `day_of_week` ; non-null `status` default `active` ; `data` .
- Index `(doctor_id,branch_id,day_of_week)` ; check weekday 0–6. There is no database uniqueness or created time. Session details are JSONB. [source: `lib/db/src/schema/core.ts:34-37`]

availability_exceptions

- `id` PK; non-null `doctor_id` -> `doctors.id`; non-null `branch_id` -> `branches.id`; non-null text `date`; non-null `status` default `active`; `data`.
- Unique `(doctor_id,branch_id,date)`. No created time and no database date-format check. [source: `lib/db/src/schema/core.ts:38-41`]

appointments

- `id` PK; non-null foreign keys `patient_id`, `doctor_id`, `clinic_id`, `branch_id`; non-null text `date`; non-null integer `token_number`; non-null `status` default `booked`; nullable `request_id`; non-null `actor_id` -> `users.id`; `data`; `created_at`.
- Unique indexes: `(doctor_id,branch_id,date,token_number)`; `(actor_id,request_id)`; expression `(data->>'reference')`; partial `(patient_id,doctor_id,branch_id,date)` when status is not terminal; partial `(doctor_id,branch_id,date)` when status is called/in consultation.
- Indexes on `patient_id` and `(clinic_id,branch_id,date)`.
- Checks: status is one of the eight lifecycle values; token number > 0. The reference uniqueness and positive-token check were added in migration `0001`.
- Date, snapshot fields, source, names, token display string, timestamps/durations, and JSON history live in `data`. [source: `lib/db/src/schema/core.ts:42-56`; `lib/db/drizzle/0001_clinicflow_integrity.sql:2,4`]

appointment_history

- `id` PK; non-null `appointment_id` -> `appointments.id`; non-null `actor_id` -> `users.id`; nullable `from_status`; non-null `to_status`; `created_at`.
- Index on `appointment_id`. It stores no action, reason, or payload snapshot. Status values are not database-checked here. [source: `lib/db/src/schema/core.ts:57-60`]

qrs

- `id` PK; non-null `clinic_id` -> `clinics.id`; nullable `branch_id` -> `branches.id`; nullable `doctor_id` -> `doctors.id`; non-null unique `public_reference`; non-null `status` default `active`; `data`; `created_at`.
- Index `(clinic_id,branch_id)`. Display name and booking URL are JSONB fields. [source: `lib/db/src/schema/core.ts:61-65`]

audit_logs

- `id` PK; nullable `actor_id` -> `users.id`; nullable `clinic_id` -> `clinics.id`; non-null `action`, `entity_type`, `entity_id`, `summary`; `created_at`.

- Index `(clinic_id,created_at)` . `entity_id` is deliberately not a foreign key because it is polymorphic. [source: `lib/db/src/schema/core.ts:66-69`]

settings

- `id` text PK; `data` JSONB. No created/update time. Runtime uses the singleton ID `platform` . [source: `lib/db/src/schema/core.ts:70` ; `artifacts/api-server/src/lib/store.ts:33-35`]

otp_challenges

- `id` PK; non-null `user_id` -> `users.id` ; non-null `mobile` ; non-null `code_hash` ; non-null timezone-aware `expires_at` ; non-null integer `attempts` default 0; nullable timezone-aware `consumed_at` ; `created_at` .
- Index `(user_id,created_at)` . No uniqueness, attempt-range check, or automatic expiry/deletion constraint. [source: `lib/db/src/schema/core.ts:71-75`]

Migration history

- `0000_clinicflow_initial.sql` creates all 15 tables, initial foreign keys/checks/indexes.
- `0001_clinicflow_integrity.sql` adds the master parent foreign key, unique appointment JSON reference, patient mobile index, and positive appointment token check.
- Drizzle schema is the current source model; no runtime startup DDL or migration runner appears in the API server. [source: `lib/db/drizzle/0000_clinicflow_initial.sql` ; `lib/db/drizzle/0001_clinicflow_integrity.sql` ; `artifacts/api-server/src/index.ts:1-25`]

9. Current implementation boundaries

These are descriptive limits visible in source:

- The OpenAPI document is the intended generated contract, but runtime resource routing is broader in two item-read cases noted above, and runtime PATCH validation is full-input rather than partial. Some response properties declared by OpenAPI (for example clinic counts, user `lastLoginAt` , schedule display names) are not populated by the generic enrichment code unless already present in JSON. [source: `artifacts/api-server/src/lib/entities.ts:3-22` ; `artifacts/api-server/src/routes/resources.ts:166-183` ; `lib/api-spec/openapi.yaml:507-515,561-571,704-713`]
- In-memory whole-table reads are used extensively. This preserves authorization before pagination but does not provide database-side pagination/aggregation and will scale with total table size. [source: `artifacts/api-server/src/lib/store.ts:12,37-52`]

- Soft deactivation does not cascade. Existing appointments/history remain, but active-context checks can block future availability and most queue transitions. There is no reactivation-specific endpoint; an authorized full PATCH can set status active.
- Weekly availability supports one effective session and one optional break per doctor/branch/day. Runtime takes the first matching active schedule if inconsistent legacy rows exist. Overnight sessions and timezone-normalized cross-branch overlaps are not supported. [source: `artifacts/api-server/src/lib/availability.ts:15-24,33-51`]
- Availability exceptions do not override all scheduling fields; queue estimates ignore exceptions; queue polling is client-driven at a fixed 30 seconds. There is no websocket/SSE route. [source: `artifacts/api-server/src/lib/availability.ts:40-51` ; `artifacts/api-server/src/routes/queue.ts:15-26`]
- General notifications, reminders, server-side export routes, clinical notes/prescriptions, payments, and file upload/storage routes are not present. Browser-side report CSV export is implemented by the frontend. Notification settings and branding/contact URLs are persisted configuration only.
- Clerk invitations are outside the DB transaction. Password recovery action does not trigger Clerk delivery. Session timeout configuration is informational to this backend. [source: `artifacts/api-server/src/routes/resources.ts:82-100,200-205` ; `artifacts/api-server/src/routes/reporting.ts:45-54`]
- Audit is a summary trail, not a before/after or security-event log. OTP events and reads are absent. Audit rows with null clinic scope are visible only to super admins through the current endpoint scope logic.
- Database status checks exist for users' role and appointments' status, but most `status` columns and JSONB domain fields rely on API validation. Direct database writers could bypass application-only clinic/branch consistency, assignment uniqueness, schedule collision, mobile/date, and many enum rules. [source: `lib/db/src/schema/core.ts:7-75`]
- Public discovery and availability intentionally require no authentication and expose active clinic/branch data, selected professional doctor fields, QR context, and capacity/session details. They do not expose user emails/mobiles through the explicit public doctor projection. [source: `artifacts/api-server/src/routes/public.ts:10-45` ; `artifacts/api-server/src/lib/entities.ts:20-22`]

10. Source map

- Server bootstrap/middleware: `artifacts/api-server/src/index.ts` , `artifacts/api-server/src/app.ts`
- Route mounting: `artifacts/api-server/src/routes/index.ts`
- Identity and scope: `artifacts/api-server/src/lib/auth.ts` , `artifacts/api-server/src/routes/identity.ts`

- Generic resources: `artifacts/api-server/src/routes/resources.ts` , `artifacts/api-server/src/lib/entities.ts`
 - Availability and booking: `artifacts/api-server/src/lib/availability.ts` , `artifacts/api-server/src/routes/appointments.ts`
 - State machine and queue locking: `artifacts/api-server/src/lib/appointments.ts` , `artifacts/api-server/src/routes/queue.ts`
 - OTP: `artifacts/api-server/src/routes/otp.ts` , `artifacts/api-server/src/lib/otp-delivery.ts`
 - Store/filter/audit/settings: `artifacts/api-server/src/lib/store.ts`
 - Reporting: `artifacts/api-server/src/routes/reporting.ts`
 - Validation/error normalization: `artifacts/api-server/src/lib/http.ts`
 - Contract and generated validation basis: `lib/api-spec/openapi.yaml` , `lib/api-zod/src/generated/api.ts`
 - Current schema/migrations: `lib/db/src/schema/core.ts` , `lib/db/drizzle/0000_clinicflow_initial.sql` , `lib/db/drizzle/0001_clinicflow_integrity.sql`
-

ClinicFlow operations and developer manual

Document basis and confidence. This manual describes the repository as inspected, not a claimed deployment state. It is based on the package manifests, workspace/build/Vite/artifact configuration, API and web sources, database schema and SQL files, operator scripts, `README.md`, `replit.md`, the existing files under `docs/`, and the attached original text specification. No application was started, no tests were run, no environment was read, and no database or identity-provider state was inspected or changed while preparing it. “Implemented in source” below therefore does not mean “runtime-verified” or “released.”

1. System boundary

ClinicFlow is a pnpm/TypeScript monorepo containing:

- a React 19/Vite web application;
- an Express 5 REST API;
- PostgreSQL access through Drizzle ORM;
- Clerk account authentication and account recovery;
- a separate, authenticated ClinicFlow mobile-number OTP flow;
- an OpenAPI contract with generated React Query and Zod code;
- development schema synchronization and operator-only bootstrap/seed scripts.

The web application and API are separate artifacts routed through the same public application. The web client calls relative `/api` URLs. API business routes use Clerk identity plus the ClinicFlow `users` record for role and tenancy checks. The implementation uses polling, not WebSocket or SSE, for queue refresh.

The repository does **not** establish that any production deployment currently exists, is current, has production secrets, has had migrations applied, or has a working SMS sender. Do not infer deployment health from committed build output.

2. Repository and source-of-truth map

Path	Purpose / authority
package.json	Root install guard and aggregate typecheck / build scripts.
pnpm-lock.yaml	Locked dependency graph.
pnpm-workspace.yaml	Workspace membership, dependency catalog, release-age policy, overrides.
tsconfig.base.json , tsconfig.json	Shared strict TypeScript settings and library project references.
.replit	Node 24 module, artifact routing/deployment target, run-button workflow name, post-merge hook, development OTP provider selection.
artifacts/clinicflow/	Production web artifact. src/App.tsx owns top-level routes/guards; src/clinic.tsx owns onboarding, portal, booking, appointment, queue, profile, settings and report flows; src/resources.tsx owns generic resource CRUD UI; src/index.css owns application styling.
artifacts/clinicflow/vite.config.ts	Vite plugins, required port/base path, build output, aliases and preview settings.
artifacts/clinicflow/.replit-artifact/artifact.toml	Web artifact route / , development command and static production output/SPA rewrite.
artifacts/api-server/	Express API artifact.
artifacts/api-server/src/app.ts	HTTP middleware order, Clerk middleware/proxy, API rate limit, mutation-origin defense, router and error handler.
artifacts/api-server/src/routes/	Actual HTTP implementations grouped as health, public discovery, identity, OTP, resources, appointments, queue and reporting.
artifacts/api-server/src/lib/	Authorization/scope, availability, queue transitions, persistence helpers, HTTP validation/errors, OTP delivery and logging.
artifacts/api-server/build.mjs	esbuild ESM bundle and Pino worker handling; outputs dist/*.mjs plus source maps.
artifacts/api-server/.replit-artifact/artifact.toml	API artifact route /api , development/build/run commands and startup health path.
lib/api-spec/openapi.yaml	API contract source of truth.
lib/api-spec/orval.config.ts	React Query and Zod generation configuration; generated API base URL is /api .
lib/api-client-react/src/generated/	Generated web client/hooks. Do not hand-edit.
lib/api-client-react/src/custom-fetch.ts	Shared fetch/error behavior and optional bearer/base-URL hooks.
lib/api-zod/src/generated/	Generated request/response validation schemas and types. Do not hand-edit.
lib/db/src/schema/core.ts	Current Drizzle schema authority.
lib/db/drizzle/0000_clinicflow_initial.sql	Initial versioned SQL artifact.

Path	Purpose / authority
lib/db/drizzle/0001_clinicflow_integrity.sql	Integrity additions: parent FK, appointment-reference uniqueness, patient-mobile index and positive-token check.
lib/db/drizzle/meta/	Drizzle snapshots and journal for the two SQL artifacts.
lib/db/drizzle.config.ts	Development Drizzle connection/schema configuration.
scripts/src/bootstrap-admin.ts	Controlled first-super-admin bootstrap.
scripts/src/seed-system.ts	Idempotent system vocabulary/settings seed; no people, clinics or appointments.
scripts/src/create-preview-accounts.ts	Explicit development/test-Clerk preview identity provisioning; not exposed as a package script.
scripts/post-merge.sh	Post-merge install/schema command; see the warning in §5.
README.md	Architecture, role/lifecycle summary and declared release boundaries.
docs/backend-interface.md	Backend integration notes and known implementation limitations.
docs/otp-operations.md	Existing OTP operational boundary.
.conversation/attached_assets/Pasted--CLINICFLOW-COMplete-NEW-SYSTEM-DEVELOPMENT-IMPORTANT-R_1790057679029.txt	Original written product specification used for the comparison in §14. It is a requirement source, not proof of implementation.
artifacts/mockup-sandbox/	Separate design/canvas artifact at /__mockup ; it is not the production ClinicFlow web artifact.

Database model at a glance

lib/db/src/schema/core.ts defines users , clinics , branches , assignments , masters , doctors , patients , schedules , availability_exceptions , appointments , appointment_history , qrs , audit_logs , settings , and otp_challenges .

Core identity, scope, queue state and token fields are relational. Several optional demographics and configuration fields are stored in JSONB data . Queue state is represented on appointments ; there is no separate queue-entry table. Important database constraints include unique Clerk/email identities, unique doctor profile per user, unique exception per doctor/branch/date, unique token per doctor/branch/date, idempotency by actor/request ID, unique appointment reference, one active patient booking per session, and at most one called/in-consultation appointment per doctor/branch/date.

3. Prerequisites and installation

Supported repository toolchain

- Node.js 24 in the Replit module configuration.

- pnpm; the root `preinstall` script rejects npm/yarn installation.
- PostgreSQL available through `DATABASE_URL`.
- Clerk configuration for authentication and identity administration.
- Twilio through the Replit connector only when real mobile OTP delivery is required.

Run commands from the repository root unless a command explicitly changes directory.

Clean deterministic install

```
pnpm install --frozen-lockfile
```

Use `pnpm install` without `--frozen-lockfile` only when intentionally changing dependencies and the lockfile. Do not create or retain `package-lock.json` or `yarn.lock`.

Exact static checks and builds

```
# Library project references, then artifact and scripts package checks
pnpm run typecheck

# Libraries only
pnpm run typecheck:libs

# Typecheck first, then every workspace package that defines build
pnpm run build

# Individual packages
pnpm --filter @workspace/api-server run typecheck
pnpm --filter @workspace/api-server run build
pnpm --filter @workspace/clinicflow run typecheck
pnpm --filter @workspace/clinicflow run build
pnpm --filter @workspace/scripts run typecheck
```

Package-script inventory (including scripts not used in the normal ClinicFlow release path):

Package	Declared scripts
root	<code>build</code> , <code>typecheck:libs</code> , <code>typecheck</code> and the pnpm-only <code>preinstall</code> <code>guard</code>
<code>@workspace/api-server</code>	<code>dev</code> , <code>build</code> , <code>start</code> , <code>typecheck</code>
<code>@workspace/clinicflow</code>	<code>dev</code> , <code>build</code> , <code>serve</code> , <code>typecheck</code>
<code>@workspace/mockup-sandbox</code>	<code>dev</code> , <code>build</code> , <code>preview</code> , <code>typecheck</code>
<code>@workspace/api-spec</code>	<code>codegen</code>
<code>@workspace/db</code>	<code>push</code> , <code>push-force</code>

Package	Declared scripts
@workspace/scripts	hello , bootstrap-admin , seed-system , typecheck
@workspace/api-client-react , @workspace/api-zod	no package scripts

Because the root recursive commands include artifact packages with matching scripts, root typechecking/building also includes the separate canvas artifact. It is not part of the production ClinicFlow web UI, but a failure there can still fail the aggregate command.

There is no test script in the inspected package manifests. A successful typecheck/build is not a substitute for the acceptance checks in §15.

API contract code generation

After any change to `lib/api-spec/openapi.yaml` , run:

```
pnpm --filter @workspace/api-spec run codegen
```

This runs Orval, cleans and regenerates both generated trees, formats generated output, then runs the root library TypeScript build. Review generated changes in:

- `lib/api-client-react/src/generated/`
- `lib/api-zod/src/generated/`

Never patch generated files as the primary change. Change the OpenAPI contract and/or generator configuration, regenerate, then update server/web consumers.

4. Environment variable inventory

This section intentionally lists **names and purposes only**. Never put values in this manual, tickets, screenshots, source, or logs.

Name	Consumer	Purpose / operational note
DATABASE_URL	lib/db and Drizzle config; API/scripts transitively	PostgreSQL connection string. Required before database-backed server or operator commands can work.
PORT	API entry point and Vite config	Listening port. API production artifact declares its port; web artifact declares its own. Vite/API fail fast if missing or invalid.
BASE_PATH	ClinicFlow and canvas Vite config	Public base path used for asset and router URLs.

Name	Consumer	Purpose / operational note
NODE_ENV	API, Vite, preview provisioning	Selects development/production behavior, logger transport, Clerk frontend proxy behavior, and development-OTP eligibility. The API dev script sets development; artifact production commands set production.
CLERK_SECRET_KEY	Clerk server SDK and production Clerk proxy; preview script gate	Server-side Clerk identity management/authentication. Preview provisioning additionally requires a Clerk test key. Secret.
CLERK_PUBLISHABLE_KEY	API Clerk middleware fallback	Server-side publishable-key fallback used with host-derived Clerk configuration. Not a secret, but still configure deliberately.
VITE_CLERK_PUBLISHABLE_KEY	Web client	Build-time web Clerk publishable-key fallback. Vite exposes VITE_ variables to client code; never put a secret in this name.
VITE_CLERK_PROXY_URL	Web client	Optional Clerk frontend API proxy URL passed to ClerkProvider ; production proxy route is /api/_clerk .
OTP_PROVIDER	API OTP delivery	Selects the mobile-OTP delivery backend. The development backend is additionally gated by the runtime environment.
SESSION_SECRET	OTP routes	HMAC secret for stored OTP challenge digests. Required for both development and Twilio mobile-verification challenges. Secret.
TWILIO_ACCOUNT_SID	OTP delivery	Twilio account identifier checked before delivery and used in connector API requests.
TWILIO_MESSAGING_SERVICE_SID	OTP delivery	Approved Twilio Messaging Service identifier/sender configuration.
LOG_LEVEL	API logger	Pino logging threshold; defaults to info .
REPL_ID	Vite	Enables Replit development-only cartographer/dev-banner plugins when present outside production.
REPLIT_DEPLOYMENT	Preview account script	Deployment guard; preview provisioning refuses the deployment environment.
CI	.replit post-build declaration	Marks the deployment post-build npm store prune environment as CI.

Do not read or print current environment values merely to “verify” setup. Confirm presence through the platform’s secret/configuration UI and validate behavior using non-sensitive health/acceptance checks.

`OTP_PROVIDER` is also declared as development-only configuration in `.replit`; that declaration does not prove any other required variable is configured.

5. Database schema operations and data safety

Development schema synchronization

The only normal schema command exposed by `@workspace/db` is:

```
pnpm --filter @workspace/db run push
```

It runs `drizzle-kit push` against `DATABASE_URL` and is documented for **development only**. Inspect the intended schema change before accepting it. Do not point this command at production.

The package also exposes:

```
pnpm --filter @workspace/db run push-force
```

This passes `--force` and can approve destructive reconciliation. It is not part of the normal procedure. Use it only for an explicitly disposable development database after review and authorization; never treat it as a recovery tool.

Versioned SQL and published environments

Two versioned SQL artifacts and corresponding Drizzle metadata are committed under `lib/db/drizzle/`. They document schema evolution. However:

- there is **no package script that applies those SQL files as a migration chain**;
- there is no migration table/runner documented in this repository;
- the API performs no startup DDL;
- existing project documentation says the managed Replit **Publish** flow applies the development-to-production schema diff;
- this manual makes no claim that a publish has occurred or that any published schema matches the current source.

For a published change, review `lib/db/src/schema/core.ts`, the committed SQL/metadata change, data compatibility, and the Publish schema diff. Do not manually run development `push` against a published database and do not add startup DDL as an operational shortcut.

Post-merge hook warning

`scripts/post-merge.sh` runs:

```
pnpm install --frozen-lockfile
pnpm --filter db push
```

The actual package name is `@workspace/db`; the filter in this hook does not match the documented/manual command. Treat the hook's schema step as **unverified and not an authoritative migration procedure**. Do not rely on it for release or recovery.

Seed semantics

Initialize system vocabulary and the singleton platform settings record with:

```
pnpm --filter @workspace/scripts run seed-system
```

The script uses conflict-safe inserts and is intended to be repeatable. It seeds workflow vocabulary, limited medical/clinic masters, and default settings. It does **not** seed people, identities, clinics, branches, schedules, patients, appointments or preview records. Existing rows are not updated by `onConflictDoNothing`; changing defaults in source will not migrate an existing settings row.

Backup and restore gap

No versioned backup script, restore script, retention policy, backup verification command, point-in-time-recovery procedure, or disaster-recovery runbook is implemented in this repository. Therefore no project-specific backup/restore command can be safely documented.

Before handling real patient data, the operator must establish a provider-approved, versioned procedure that covers at minimum:

1. scheduled encrypted database backups;
2. retention and access control;
3. backup version/schema identification;
4. restore into an isolated environment;
5. integrity and application smoke validation after restore;
6. Clerk/Twilio external-system reconciliation;
7. periodic restore drills and recorded recovery objectives.

Do not improvise `pg_dump` / `psql` production commands from this manual; ownership, credentials, managed-service constraints and recovery targets are not defined in the repository.

6. Running the services

Development

The configured artifact commands are:

```
# API: selects development runtime behavior, rebuilds, then starts the bundle
pnpm --filter @workspace/api-server run dev

# Web Vite development server
pnpm --filter @workspace/clinicflow run dev
```

The API development artifact is routed under `/api`. The web artifact uses its declared service port and base path. Avoid hardcoding local ports in client code; generated clients use relative `/api`.

The API `dev` script is build-and-start, not watch mode. Source changes require another build/start cycle by the owning workflow/operator.

Useful direct commands:

```
# Run an already-built API bundle
pnpm --filter @workspace/api-server run start

# Preview an already-built web bundle with Vite
pnpm --filter @workspace/clinicflow run serve
```

Production artifact contract

- API build: `pnpm --filter @workspace/api-server run build`
- API run: `node --enable-source-maps artifacts/api-server/dist/index.mjs`
- API startup health path: `/api/healthz`
- Web build: `pnpm --filter @workspace/clinicflow run build`
- Web output: `artifacts/clinicflow/dist/public`
- Web serving: static, with `/*` rewritten to `/index.html` for SPA routes

The repository also has a root `pnpm run build`, which runs typechecking before package builds. Prefer that aggregate gate for release candidates.

Do not infer current publish status from `.replit-artifact/artifact.toml`; it describes the artifact contract only.

7. Authentication, account creation and setup journeys

Two distinct verification systems

Clerk account authentication

- Owns sign-up, sign-in, password handling, recovery, sessions and logout.
- Establishes the Clerk identity consumed by API middleware.
- Requires a verified primary email for ClinicFlow onboarding.
- Test-instance email verification may use Clerk's documented test-address convention: an address containing `+clerk_test`, with verification code `424242`. This is for a Clerk **development/test instance only**. Never use that convention or code in production.
- ClinicFlow does not store or hash account passwords itself because password authentication is delegated to Clerk.

ClinicFlow mobile OTP

- Is a separate challenge available only after Clerk sign-in and ClinicFlow onboarding.
- Verifies the mobile attached to the current ClinicFlow patient/user.
- Uses `POST /api/otp/request` and `POST /api/otp/verify`.
- Stores an HMAC digest, not the plaintext code, and applies expiry, resend, hourly request, attempt and single-use controls.
- In development-provider mode, the API returns the newly generated `developmentCode`; it is not the Clerk code `424242` and proves no ownership of the phone number.
- In Twilio mode, the code is sent through the connector and is never returned by the API.

Never confuse Clerk's fixed test-instance email-verification code with the randomized ClinicFlow mobile OTP.

Ordinary patient setup

1. Open `/register` (redirects to Clerk `/sign-up`).
2. Complete Clerk sign-up and verify the primary email.
3. The application redirects to `/onboarding`.
4. With no explicit doctor intent, onboarding creates a ClinicFlow `patient` user and linked patient profile.
5. Complete/update profile and mobile details.
6. If platform settings require mobile verification, complete the separate mobile OTP before booking.
7. Book from `/patient/book` or a valid `/book/:reference`; accept booking consent; retain the returned reference/token.

Doctor self-registration

1. Start at `/register-doctor`. This stores a session-scoped doctor intent and redirects to Clerk sign-up.
2. Verify the primary Clerk email.
3. `/onboarding` creates a ClinicFlow `doctor` user and linked doctor profile.
4. Complete professional profile details.
5. Create a clinic where permitted or receive clinic/branch assignments from an administrator.
6. Create branches if the doctor owns the clinic.
7. Configure weekly schedule, capacity, token prefix, queue mode and optional date exceptions.
8. Only active, assigned doctors with an open, available branch session become bookable.

Doctor intent is held in browser `sessionStorage`. Beginning a normal registration instead creates a patient. Existing users cannot self-switch roles.

Invited staff and managed users

Super administrators, and clinic administrators within their restrictions, can create managed users. The server either links an existing verified Clerk identity or creates a Clerk invitation, then writes the ClinicFlow profile/assignments. A Clerk invitation and the PostgreSQL transaction cannot be atomic: if the database write fails after invitation creation, an unassigned invitation may remain and needs operator reconciliation.

The “password reset” administrative API records an audit event and tells the user to use Clerk’s Forgot password flow. It does not send recovery mail itself.

First super administrator bootstrap

There is no public super-admin claim and the system seed does not create an administrator.

Prerequisites:

- the intended operator identity already exists in Clerk;
- its primary email is verified;
- it has not completed patient/doctor onboarding;
- `DATABASE_URL` and server-side Clerk configuration are available to the command;
- no active super administrator exists.

Preferred command:

```
pnpm --filter @workspace/scripts run bootstrap-admin -- --clerk-id <verified-clerk-user-id>
```

The script also supports verified-primary-email lookup:

```
pnpm --filter @workspace/scripts run bootstrap-admin -- --email <verified-primary-email>
```

Prefer immutable Clerk ID to avoid ambiguous operational input. The script:

- looks up the Clerk identity and requires its verified primary email;
- takes a PostgreSQL advisory transaction lock;
- refuses to run if an active super administrator exists;
- refuses silent promotion of an existing non-admin ClinicFlow profile;
- creates/activates the super-admin record and writes an audit event;
- generates no password and changes no Clerk credentials.

After success, sign in through Clerk with the same identity. There is no supported “rerun to promote another user” path; use authorized administration while retaining last-active-super-admin protection.

Development-only preview accounts

`create-preview-accounts.ts` is operator-only and is never run at startup/deploy. It has no package alias, so the exact repository command is:

```
pnpm --filter @workspace/scripts exec tsx ./src/create-preview-accounts.ts
```

Safety gates require all of the following:

- the runtime environment is configured as development;
- a Clerk test-instance secret key;
- not running in a Replit deployment environment;
- none of the fixed preview profiles already exists in PostgreSQL;
- none of the matching identities already exists in Clerk.

It provisions one identity/profile for each supported role, plus linked doctor/patient records where applicable. It generates strong random passwords, prints the newly generated credentials once to the invoking operator, and does not store passwords in project files. Do not paste that output into this manual, source control, chat, tickets or logs. Store it only in an approved development credential channel, then clear terminal history/output according to local policy.

The generated test addresses use Clerk’s `+clerk_test` convention. When an interactive Clerk test-instance email challenge is presented, use Clerk’s development-only code `424242`; do **not** use that code for ClinicFlow mobile OTP. Never run this procedure against a production Clerk instance or published environment.

The script attempts to delete newly created Clerk identities if later provisioning fails, but cleanup itself can fail; follow any emitted identity IDs in the Clerk test administration UI. The script intentionally refuses updates and is not a reset command. Remove preview identities/data using an explicitly reviewed development cleanup process; none is implemented here.

8. Routing, services and authorization

Web routes

Public/authentication routes:

- `/` — landing page; signed-in users are redirected to onboarding/role resolution.
- `/sign-in/*` and `/sign-up/*` — Clerk components.
- `/login`, `/register`, `/forgot-password` — redirects into Clerk routes.
- `/register-doctor` — records doctor intent, then redirects to sign-up.
- `/onboarding` — authenticated ClinicFlow profile creation.
- `/book/:reference` — resolves a revocable public QR reference; sign-in/onboarding is required before booking.

Protected portal routes:

- Admin shell:
`/admin/{dashboard,clinics,branches,doctors,users,patients,masters,appointments,queue,reports,settings,audit,qrs,book}`
- Doctor:
`/doctor/{dashboard,profile,clinics,branches,availability,exceptions,appointments,queue,patients,qrs,book}`
- Receptionist: `/receptionist/{dashboard,appointments,queue,patients,book}`
- Patient: `/patient/{dashboard,book,appointments,queue,profile}`

`superAdmin` and `clinicAdmin` share the `/admin` shell, but clinic admins are redirected away from users, masters, settings and audit pages. Client guards are user experience only; API authorization remains authoritative.

API surface

All API routes are under `/api`. The OpenAPI operation inventory is:

Family	Methods and paths
Health	GET <code>/healthz</code>

Family	Methods and paths
Identity	GET/PATCH /me ; POST /onboarding
Mobile OTP	POST /otp/request ; POST /otp/verify
Public discovery	GET /public/clinics ; /public/branches ; /public/doctors ; /public/availability ; /public/qr/{reference}
Clinics	GET/POST /clinics ; GET/PATCH/DELETE /clinics/{id}
Branches	GET/POST /branches ; GET/PATCH/DELETE /branches/{id}
Doctors	GET/POST /doctors ; GET/PATCH/DELETE /doctors/{id}
Users	GET/POST /users ; GET/PATCH/DELETE /users/{id} ; POST /users/{id}/password-reset
Patients	GET/POST /patients ; GET/PATCH/DELETE /patients/{id}
Masters	GET/POST /masters ; GET/PATCH/DELETE /masters/{id}
Schedules	GET/POST /schedules ; PATCH/DELETE /schedules/{id}
Date exceptions	GET/POST /availability-exceptions ; PATCH/DELETE /availability-exceptions/{id}
Appointments	GET/POST /appointments ; GET /appointments/{id} ; POST /appointments/{id}/actions
Queue	GET /queue ; POST /queue/call-next
QR	GET/POST /qrs ; GET/PATCH/DELETE /qrs/{id} ; POST /qrs/{id}/regenerate
Dashboard/reporting	GET /dashboard ; GET /reports ; GET /audit-logs
Settings	GET/PATCH /settings

For request/response bodies, query parameters, enums and response codes, use `lib/api-spec/openapi.yaml` ; do not infer payloads from this summary.

Request handling and security controls in source

- Pino request logging runs first and redacts authorization, cookie and set-cookie headers.
- Helmet is enabled, but its content-security-policy feature is explicitly disabled.
- JSON bodies are limited to 128 KiB.
- Clerk middleware establishes identity.
- `/api` has a general 180 requests/minute limiter.
- `/otp` has an additional 30 requests/15 minutes limiter.
- Mutating methods reject cross-site browser requests and require either matching origin/host or bearer authentication.
- Validation uses generated Zod schemas plus date/mobile checks.

- Known validation/conflict errors receive controlled messages; unexpected server errors are logged and returned as a generic service-unavailable response.
- Public endpoints expose active discovery data and QR context, not patient data.

Roles and tenancy

- `superAdmin` : global scope.
- `clinicAdmin` : assigned clinic scope; cannot grant administrator roles and does not receive platform users/masters/settings/audit UI.
- `doctor` : own doctor context and relevant assigned/owned clinic, schedule, appointment, patient and queue data.
- `receptionist` : assigned clinic/branch scope.
- `patient` : own profile, own appointments and own queue projection.

API checks are implemented in `src/lib/auth.ts` and per-route write authorization. Assignment validation ensures selected branches belong to selected clinics. Collection reads currently load records, enrich them, authorize/filter them, then sort/page them in the server process.

9. Core service behavior

Availability

Availability requires an active doctor account/profile, active clinic/branch, branch assignment, an open weekly schedule, no closing exception, a non-past session, a date inside the booking horizon, and remaining token capacity. Weekly schedules support one session with one optional break per doctor/branch/day. Overnight sessions are rejected. Different timezones across potentially overlapping branch schedules are conservatively rejected.

Date exceptions can close a date or override times/break/capacity. An open-time override requires an existing weekly schedule. The implementation does not model multiple sessions in one branch/day.

Booking and tokens

Booking is revalidated inside a database transaction using advisory locks. The service validates actor scope, patient status/mobile, doctor availability, queue mode, capacity, optional mobile-verification policy, consent for patient bookings, and QR context. Request IDs provide actor-scoped idempotency. Tokens are allocated sequentially per doctor/branch/date and protected by unique constraints.

Supported source codes are `online`, `walkIn`, `phone`, and `qr`; the current portal booking UI submits staff-created bookings as `walkIn`. Walk-ins are restricted to today and queue-open/break rules. The UI does not expose an explicit staff selector distinguishing phone from walk-in.

Queue lifecycle

Implemented transitions:

```
booked -> checkedIn -> waiting -> called -> inConsultation -> completed
waiting|called -> noShow -> waiting
booked|checkedIn|waiting -> cancelled
```

Transitions are validated, scoped, history/audit recorded, and protected with queue locks. Queue actions other than cancellation occur only on the appointment's local date. A doctor/branch/date cannot have another `called` or `inConsultation` record. `call-next` picks the oldest waiting record.

Queue views refresh every 30 seconds. Staff receive entries; patients receive aggregate values and their own entry only. Estimated wait is patients ahead multiplied by configured consultation plus buffer minutes; it is an estimate, not a prediction model.

QR booking

QR records contain a random public reference and clinic with optional branch/doctor context. The web UI renders/downloads PNGs. Public resolution returns context only. Regeneration replaces the reference, invalidating printed/old links. Deactivation uses the generic inactive status path. A QR booking is matched to the locked QR context again at booking time.

Audit and reports

Important CRUD, onboarding, booking, transitions, QR regeneration, settings and bootstrap events write `audit_logs`. Reports group scoped appointment metrics by date, clinic or doctor and can export CSV in the browser. Dashboard/report calculations are performed from persisted rows in the API process.

Audit records currently contain actor, clinic scope, action, entity type/ID, summary and timestamp. They do not implement the original specification's complete old value, new value or client IP record.

10. Mobile OTP and Twilio status

Development provider

Configure the environment variable names documented in §4 for the development runtime and development OTP backend. `SESSION_SECRET` is still required. A request returns a randomized `developmentCode` only when both development gates are active. The code exercises digest, expiration, resend, request-limit, attempt-limit and one-time-consumption logic, but does not deliver a message and does not prove possession of the phone.

Never expose development-provider responses publicly and never configure development OTP in production. Production mode cannot enable it.

Twilio provider

Twilio mode requires:

- the OTP backend selector named in §4 to select the production SMS backend;
- the Twilio account and Messaging Service variables named in §4;
- an authorized Replit Twilio connector;
- an approved sender/service with appropriate regional permissions;
- `SESSION_SECRET` .

The application submits through `@replit/connectors-sdk` , reports unavailable/rejected delivery as an explicit 503, and does not silently fall back to development verification.

Current documented status: production SMS integration is pending and has not been live-delivery tested. Do not mark mobile OTP production-ready until sender authorization, delivery, failure modes, regional restrictions, recovery and the end-to-end patient booking journey are verified in the target environment.

OTP behavior in source:

- mobile format is international E.164-like (`+` plus 8–15 digits);
- expiry is setting-driven but clamped to 60–900 seconds;
- resend cooldown is 60 seconds;
- maximum five requests/account/hour;
- route limit is 30 requests/15 minutes;
- attempts are setting-driven but clamped to 1–10;
- a new challenge consumes prior challenges;
- successful verification updates patient and user mobile state transactionally;
- changing a patient mobile resets verification.

11. Observability and troubleshooting

Available observability

- `GET /api/healthz` is the configured API startup probe.
- The API emits Pino logs. Development uses `pino-pretty` ; production uses structured output.
- `LOG_LEVEL` controls severity.

- HTTP log serializers include request ID, method, URL path without query string, and response status.
- Sensitive authorization/cookie response/request headers are redacted.
- Unexpected request failures are logged server-side; clients receive a generic message.
- Source maps are generated and enabled by the production API command.
- Audit events provide business-operation history, not infrastructure telemetry.

There is no repository implementation for metrics, distributed traces, uptime alerting, log retention, dashboards, paging, error aggregation or audit export retention.

Troubleshooting sequence

1. **Separate infrastructure from identity from business rules.** Check `/api/healthz` ; then Clerk sign-in; then `/api/me` ; then the specific business endpoint.
2. **API will not start:** confirm that `PORT` is configured and valid, database configuration is available, and the API bundle exists. Rebuild with the package build command. Do not inspect or print secret values.
3. **Web Vite will not start/build:** confirm `PORT` and `BASE_PATH` names are configured; ensure the lockfile install completed; typecheck the web package.
4. **401 "Sign in required":** the Clerk session/token did not reach the API. Check same-origin routing and Clerk client/proxy configuration names.
5. **403 "Complete onboarding first":** Clerk identity exists but no ClinicFlow user is linked. Complete `/onboarding` , accept an invitation with the matching verified email, or use the controlled first-admin procedure as appropriate.
6. **403 "Account inactive" / permission denied / outside scope:** inspect the user's persisted status, role and clinic/branch assignments through authorized admin UI. Do not bypass API authorization.
7. **Clerk invitation succeeded but profile creation failed:** because Clerk and PostgreSQL are not transactional together, reconcile the invitation and database record before retrying.
8. **No public clinics/doctors:** only active records are returned; doctors also need active branch assignments. Create/activate real records and schedules rather than fixtures.
9. **Availability says unavailable:** check doctor/account/clinic/branch active states, assignment, weekday schedule, date exception, local timezone/date, booking horizon, session end and remaining capacity.
10. **Schedule conflict:** only one branch/day session is supported, overlaps are rejected, and differing-timezone cross-branch schedules are treated conservatively.
11. **Booking 409:** refresh availability and inspect active duplicate booking, capacity, queue mode, closing time, QR context and idempotency-key reuse.
12. **Queue action 409:** refresh first; verify expected state, local appointment date and whether another entry is called/in consultation.

13. **Patient cannot view queue:** the selected appointment must belong to the signed-in patient and match the requested doctor/branch/date.
14. **OTP unavailable:** confirm configuration names and mode, `SESSION_SECRET`, then provider readiness. In Twilio mode inspect connector/provider authorization without logging code/body payloads.
15. **OTP throttled/expired:** observe the 60-second resend cooldown, hourly/request route limits, configured attempts and expiry; request a new challenge rather than altering persisted counters.
16. **Old QR fails:** regeneration/deactivation is intentionally revocable. Generate/distribute the current QR.
17. **Lists slow at scale:** current list/report logic is in-process after broad database reads. This is an architectural limit, not a browser-cache problem.
18. **Stale UI after account switch:** the web client clears React Query cache on Clerk user changes; if behavior persists, sign out fully and verify the active Clerk identity rather than changing roles in data.

Never log OTP bodies/codes, credentials, cookies, bearer tokens, database URLs, Clerk keys or preview-account output.

12. Known operational and product limitations

1. Production Twilio SMS delivery is pending and unverified.
2. Notifications beyond mobile OTP and Clerk invitations are not implemented. Persisted notification settings do not deliver reminders, email, WhatsApp or in-app notifications.
3. Clerk governs session lifetime. `sessionTimeoutMinutes` in settings is informational and does not reconfigure Clerk.
4. Collection authorization/filtering/sorting/pagination and report/dashboard aggregation happen in server memory after broad reads. SQL-side queries are needed before larger deployments.
5. One weekly session and one optional break per doctor/branch/weekday; no split sessions or overnight sessions.
6. Cross-branch overlap checks with differing timezones are conservative.
7. Clerk invitations and PostgreSQL writes cannot be atomic.
8. Queue "live" behavior is 30-second polling, not push.
9. There is no repository test suite/script or recorded acceptance evidence.
10. There is no implemented backup/restore runbook or automated restore verification.
11. There is no committed migration runner for applying versioned SQL; development uses schema push and published schema changes are delegated to Replit Publish.
12. Generic resource UI supports search and pagination for many lists, but not every page exposes all API filters or a sort control.

13. The generic UI hardcodes some workflow/form choices (for example roles, status, gender, queue mode) even though masters exist. This does not fully meet the original “all master values dynamic” requirement.
14. Seeded masters are a limited vocabulary. Country/state/city/area/pincode records are not populated by `seed-system`.
15. Administrative password-reset action only directs the user to Clerk recovery; it does not trigger a reset email.
16. The staff booking screen does not explicitly distinguish phone booking from walk-in; current staff portal submission uses `walkIn`.
17. Audit events do not preserve complete old/new snapshots or request IP.
18. The database has no separate roles, permissions, user-role, queue-entry or notification tables; role is a constrained user field and queue state resides on appointments.
19. Some optional domain fields/configuration live in JSONB rather than fully normalized columns.
20. Content Security Policy is disabled in Helmet configuration; security review is required.
21. No healthcare regulatory compliance claim is made. Security, privacy, retention and legal review remain release prerequisites.
22. The repository documentation says the separate visual wireframe was not available as an implementation reference; pixel-level conformance is unverified.
23. Preview account creation has no package script alias or cleanup command.
24. The post-merge database filter is inconsistent with the actual package name and must not be relied on.

13. Change procedures

API contract or endpoint change

1. Update `lib/api-spec/openapi.yaml`.
2. Run the codegen command.
3. Implement/update the relevant API router and authorization.
4. Update web consumers.
5. Run typecheck/build gates.
6. Review generated diff for unexpected removals and validate role/tenancy behavior in acceptance testing.

Database change

1. Update `lib/db/src/schema/core.ts`.

2. Create/review a versioned SQL and Drizzle metadata change using the project's approved schema workflow; do not handwave a temporary runtime DDL script.
3. Review backward compatibility, constraints and existing data.
4. Use `push` only against development.
5. Update OpenAPI/server/web and seed logic as needed.
6. For published environments, review the managed Publish diff and record approval; do not claim it was applied without deployment evidence.
7. Ensure backup/restore readiness before destructive production changes—currently a documented gap.

Role/authorization change

Review together:

- schema role constraint;
- `auth.ts` scope/can-read rules;
- route write authorization;
- OpenAPI role enums/descriptions;
- generated clients/Zod;
- frontend route map/navigation/guards;
- `seed-system.ts` governed vocabulary;
- bootstrap/preview scripts;
- cross-role negative acceptance cases.

Frontend hiding is never sufficient.

14. Original specification compared with the inspected project

Status vocabulary:

- **Source evidence** — implementation is present in inspected source, but runtime behavior was not tested for this manual.
- **Partial** — some required behavior exists, with a material omission or narrower implementation.
- **Unmet/gap** — required behavior/procedure is absent in inspected source.
- **Unverified** — cannot be established without running acceptance/security/deployment checks or having the referenced design evidence.

No row should be interpreted as a release certification.

Original specification area	Status against inspected project
New React/TypeScript, Node REST, PostgreSQL architecture	Source evidence. Separate web/API artifacts, Drizzle schema and generated contract exist.
Real authentication, logout, recovery	Source evidence / unverified runtime. Clerk owns these flows. Password storage/hashing is delegated rather than implemented locally.
Genuine backend RBAC and tenancy	Source evidence / unverified security. API role/scope checks exist for all five roles; penetration and multi-account negative tests remain unverified.
No production role switcher	Source evidence. No role-switching control is present; roles come from persisted users.
Super admin	Source evidence / unverified journey. Admin shell, APIs and controlled bootstrap exist.
Clinic admin	Partial. Real constrained role/scope exists, but it shares the admin shell and is intentionally excluded from users, masters, settings and audit; full original clinic-admin matrix has not been demonstrated.
Doctor registration/profile/clinic management	Source evidence / unverified journey. Doctor intent onboarding, profile and owned/assigned clinic behavior exist.
Receptionist login/dashboard/assigned scope	Source evidence / unverified journey. Clerk login is shared rather than a dedicated authentication backend; scoped pages/actions exist.
Patient registration/profile	Source evidence / unverified journey.
User management fields/actions	Partial. CRUD/status/role/assignment and Clerk invitation exist. Last-login data is absent; reset action provides recovery instructions only; existing role switching is intentionally blocked.
Master management and dynamic usage	Partial/unmet strict criterion. CRUD/search/pagination and seeded vocabulary exist, but several UI/workflow enums remain hardcoded and location masters are not seeded.
Clinic and branch CRUD	Source evidence / unverified journey. Deletion is status deactivation.
Doctor management and assignments	Source evidence / unverified journey.
Weekly availability, break, capacity, clinic-specific schedule	Source evidence. Limited to one session/one break/day and no overnight sessions.
Leave/holiday/modified-time exceptions	Source evidence. Date exceptions support close or overrides.
Queue settings	Partial. Capacity, prefix, queue mode, duration, buffer and queue open/close exist per schedule; broader standalone settings/master behavior is narrower.
Appointment, walk-in, phone, QR, online sources	Partial. API source enum supports all; UI supports online/QR and submits staff booking as walk-in, without an explicit phone-booking choice.
Multi-step appointment booking with real availability	Source evidence / unverified concurrency journey.

Original specification area	Status against inspected project
QR generate/view/download/regenerate/deactivate and functional booking	Source evidence / unverified scan journey.
Patient search by required fields/history	Partial. Generic search covers name/mobile/code/email and appointment listing exists, but the exact combined result/history screen is not established.
Queue state machine, call next, no-show, requeue	Source evidence / unverified concurrent use.
Patient aggregate queue, token, ahead, estimate, auto-refresh	Source evidence. Uses 30-second polling and formula-based estimate.
Dynamic dashboards	Source evidence / unverified accuracy. Values derive from persisted API data; role presentation is shared and not every original metric is shown.
Reports	Source evidence / partial. Scoped grouped metrics and CSV export exist; not every originally listed report is a separate report.
Settings	Partial. Core platform/booking/OTP/terms fields exist; branding/notifications are not connected, and session timeout cannot control Clerk.
Audit logging	Partial. Important events are logged; full old/new/IP details are absent.
Database entities, FKs and indexes	Partial. Core relational model and integrity indexes exist; separate permissions, queue entries and notifications are absent; optional data uses JSONB.
Generated IDs/references/tokens	Source evidence. Backend generates IDs, codes, random appointment references and sequential tokens; exact human-readable formats differ from examples in the specification.
Search/filter/sort/pagination on all major tables	Partial. API supports shared filtering/sorting/pagination, but processing is in memory and UI controls are not exhaustive.
Empty/loading/error/retry states	Partial. Common states exist; retry is not consistently exposed on every API page.
Notifications	Unmet/gap beyond Clerk invitations and OTP.
Near-real-time synchronization	Partial. Controlled 30-second polling is implemented; no WebSocket/SSE.
Responsive/mobile experience and accessibility	Source evidence / unverified. Responsive CSS/components and labels/focus semantics are present, but WCAG and device acceptance were not run.
Security controls	Partial/unverified. Clerk, validation, parameterized ORM, rate limits, same-origin mutation checks, secure QR references, audit and secret-based config exist. CSP is disabled; no security assessment was run.
Development seed/demo accounts	Partial. System seed is non-demo; explicit development preview provisioning exists but is not automatic and credentials are deliberately not documented. This safely diverges from the request to publish passwords in README.

Original specification area	Status against inspected project
Migrations/reproducible database	Partial. Versioned SQL artifacts exist, but no migration-apply package command is implemented; development uses Drizzle push.
README/API/deployment documentation	Partial. README and OpenAPI exist; this manual expands operations. Backup/restore and definitive deployment-state evidence remain absent.
Exact wireframe/screen conformance	Unverified. Project documentation states the separate visual wireframe was not available; the written journeys were used.
Production-ready claim	Unmet/unverified. SMS, backup/restore, scale, security, privacy/compliance and full acceptance testing are outstanding.

15. Acceptance and release checklist

This is a checklist to execute and record before release; it is **not** a statement that tests have been run.

Build and contract

- ☐ Clean `pnpm install --frozen-lockfile` succeeds with the approved Node/pnpm toolchain.
- ☐ `pnpm run typecheck` succeeds.
- ☐ `pnpm run build` succeeds.
- ☐ Generated clients/Zod match the reviewed OpenAPI contract; no hand-edited generated files.
- ☐ Database schema diff, versioned SQL/metadata and compatibility review are approved.
- ☐ No secrets, preview credentials, patient data or OTPs appear in source, build output or logs.

Environment and operations

- ☐ Required environment variable **names** are configured in the target environment through approved secret/configuration management.
- ☐ Development-only OTP and preview provisioning are disabled in every published environment.
- ☐ Health probe succeeds through the public router.
- ☐ Structured logs, retention, access, alerting and incident ownership are configured.
- ☐ Database backup policy exists and a restore drill has succeeded against the exact release schema.
- ☐ Rollback/data-recovery decision points are documented.
- ☐ Published schema application evidence is recorded; no speculative “migration complete” claim.

Identity and authorization

- [] Sign-up, verified email, sign-in, logout and Clerk recovery work.
- [] Patient and doctor onboarding create the correct immutable role/profile.
- [] First-admin bootstrap is exercised only in an isolated acceptance environment.
- [] Super admin, clinic admin, doctor, receptionist and patient each see only intended routes/data/actions.
- [] Cross-clinic, cross-branch, cross-doctor and cross-patient API access is denied even with direct requests.
- [] Inactive accounts and records are denied appropriately.
- [] Last-active-super-admin protection works.
- [] Invitation/database partial-failure reconciliation is rehearsed.

Core end-to-end journey

- [] Super admin creates a clinic and branch and assigns staff.
- [] Doctor signs in, completes profile, receives/creates permitted clinic context, and configures weekly availability.
- [] Date closures and modified-time exceptions affect discovery/booking.
- [] Receptionist sees only assigned doctors/patients and registers/searches a patient.
- [] Patient sees only active, assigned, available options.
- [] Online and walk-in bookings generate unique references/tokens; phone source behavior is either implemented or explicitly excluded from release.
- [] Capacity, duplicate booking, booking horizon, queue mode, queue open/close, break and concurrency conflicts are enforced.
- [] QR generation/download/open, context restriction, booking, regeneration invalidation and deactivation work.
- [] Check-in, enqueue, call-next, start, complete, no-show, requeue and cancellation transitions work; invalid/stale transitions fail.
- [] Doctor, receptionist and patient polling views converge; patient data never appears in another patient's response.
- [] Dashboard/report/audit values reflect persisted changes.

Mobile OTP and communications

- [] Development mobile OTP is tested only in a non-public development environment and is clearly labeled.
- [] Clerk `+clerk_test / 424242` testing is kept distinct from ClinicFlow mobile OTP.
- [] Twilio sender/service is approved and connector access is authorized.

- [] Real SMS delivery, expiration, cooldown, hourly limits, attempt limits, replacement and single use work in the target region.
- [] Provider outage/rejection returns an explicit failure and never bypasses verification.
- [] Changing mobile resets verification.
- [] If Twilio remains pending, production mobile-verification-dependent workflows are not released as complete.
- [] Notification features not implemented are not advertised.

Quality, security and release decision

- [] Original nine end-to-end acceptance scenarios are executed with independent real accounts and recorded evidence.
- [] Concurrent booking/token and queue-transition tests pass.
- [] Validation, rate-limit, same-origin/CSRF, XSS, dependency and authorization security review passes.
- [] CSP decision is reviewed and documented.
- [] Responsive desktop/tablet/mobile and keyboard/accessibility checks pass.
- [] Empty, loading, error, retry and unavailable-provider states are reviewed.
- [] In-memory list/report performance is measured with expected production volume or release is constrained accordingly.
- [] Privacy, retention, audit, healthcare/legal and incident-response reviews approve the intended data use.
- [] Every item classified partial, unmet or unverified in §14 is closed or explicitly accepted as a release limitation by the accountable owner.

16. Primary source pointers

- Commands/workspace: `package.json` ; `pnpm-workspace.yaml` ; each package's `package.json` .
- Artifact lifecycle: `.replit` ; `artifacts/*/replit-artifact/artifact.toml` .
- Web route/role maps: `artifacts/clinicflow/src/App.tsx` (`routes` , `Guard`);
`artifacts/clinicflow/src/clinic.tsx` (`navConfig` , `Onboarding` , `Booking` , `Queue`).
- Resource UI and hardcoded choices: `artifacts/clinicflow/src/resources.tsx` (`resources` , `ResourcePage`).
- API middleware/security: `artifacts/api-server/src/app.ts` ; `src/lib/http.ts` ;
`src/lib/logger.ts` .
- Authentication/tenancy: `artifacts/api-server/src/lib/auth.ts` ; `src/routes/identity.ts` ;
`src/routes/resources.ts` .

- Availability/booking/queue: `src/lib/availability.ts` ; `src/routes/appointments.ts` ; `src/lib/appointments.ts` ; `src/routes/queue.ts` .
 - OTP/Twilio: `src/routes/otp.ts` ; `src/lib/otp-delivery.ts` ; `docs/otp-operations.md` .
 - API contract/codegen: `lib/api-spec/openapi.yaml` ; `lib/api-spec/orval.config.ts` .
 - Schema/SQL: `lib/db/src/schema/core.ts` ; `lib/db/drizzle/` ; `lib/db/package.json` .
 - Operator scripts: `scripts/src/bootstrap-admin.ts` ; `scripts/src/seed-system.ts` ; `scripts/src/create-preview-accounts.ts` .
 - Declared limitations: `README.md` ("Release boundaries"); `docs/backend-interface.md` .
 - Original requirement baseline: `.conversation/attached_assets/Pasted--CLINICFLOW-COMPLETE-NEW-SYSTEM-DEVELOPMENT-IMPORTANT-R_1790057679029.txt` .
-

Appendix: Exact OpenAPI contract

This appendix reproduces the current interface definition from `lib/api-spec/openapi.yaml`. It includes every path, operation, parameter, request schema, response schema, field, enumerated value, required-field list, and declared constraint in that file. It is an interface reference, not evidence that every behavior has passed runtime acceptance testing. For implementation caveats, use the preceding chapters.

```
openapi: 3.1.0
info:
  title: Api
  version: 1.0.0
  description: |
    ClinicFlow REST contract. Clerk identity is mapped to a database role.
    Super admins access all; clinic admins and receptionists are assignment-scoped;
    doctors access their own resources and relevant patients; patients access self only.
    All filters narrow authorized scope, never expand it. DELETE deactivates records
    with history. Lists contain real persisted data only. Queue clients poll every 30 seconds.
    Invalid input returns 400, unauthenticated 401, forbidden 403, missing 404,
    state/capacity conflicts 409, rate limiting 429, unavailable integrations 503.
servers:
  - url: /api
security:
  - ClerkBearer: []
paths:
  /healthz:
    get:
      operationId: healthCheck
      security: []
      responses:
        "200": { description: Healthy, content: { application/json: { schema: { $ref:
"#/components/schemas/HealthStatus" } } } } }
  /me:
    get:
      operationId: getMe
      description: Returns database identity; user is null before onboarding. Never accepts a role from the
client.
      responses:
        "200": { description: Identity, content: { application/json: { schema: { $ref:
"#/components/schemas/Identity" } } } } }
    patch:
      operationId: updateMe
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/ProfileInput" } } } } }
      responses:
        "200": { description: Profile updated, content: { application/json: { schema: { $ref:
"#/components/schemas/User" } } } } }
  /onboarding:
    post:
      operationId: onboard
      description: Atomically creates the first profile only. Default intent patient; doctor requires explicit
```

```
intent. Existing profiles return 409; no admin self-claim or role switching.
  requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/OnboardingInput" } } } }
  responses:
    "201": { description: Onboarded, content: { application/json: { schema: { $ref:
"/components/schemas/Identity" } } } }
    "409": { $ref: "#/components/responses/Error" }
/otp/request:
  post:
    operationId: requestOtp
    description: Rate-limited mobile challenge bound to authenticated identity. External SMS provider required
in production. Never returns the code in production; an explicitly enabled provider with NODE_ENV=development
returns developmentCode only in development.
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/OtpRequest" } } } }
    responses:
      "200": { description: Challenge sent, content: { application/json: { schema: { $ref:
"/components/schemas/OtpChallenge" } } } }
      "503": { $ref: "#/components/responses/Error" }
/otp/verify:
  post:
    operationId: verifyOtp
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/OtpVerification" } } } }
    responses:
      "200": { description: Mobile verified, content: { application/json: { schema: { $ref:
"/components/schemas/OtpResult" } } } }
/public/clinics:
  get:
    operationId: listPublicClinics
    security: []
    parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/Page" }, { $ref:
"/components/parameters/PageSize" }]
    responses:
      "200": { description: Active public clinics, content: { application/json: { schema: { $ref:
"/components/schemas/ClinicList" } } } }
/public/branches:
  get:
    operationId: listPublicBranches
    security: []
    parameters: [{ $ref: "#/components/parameters/ClinicId" }, { $ref: "#/components/parameters/Page" }, {
$ref: "#/components/parameters/PageSize" }]
    responses:
      "200": { description: Active branches, content: { application/json: { schema: { $ref:
"/components/schemas/BranchList" } } } }
/public/doctors:
  get:
    operationId: listPublicDoctors
    security: []
    parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { name: specializationId, in: query, schema: { type: string } }, {
$ref: "#/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }]
    responses:
      "200": { description: Public professional profiles only, content: { application/json: { schema: { $ref:
"/components/schemas/PublicDoctorList" } } } }
/public/availability:
  get:
```

```

    operationId: getPublicAvailability
    security: []
    parameters:
      - { name: doctorId, in: query, required: true, schema: { type: string } }
      - { name: branchId, in: query, required: true, schema: { type: string } }
      - { name: date, in: query, required: true, schema: { type: string, format: date } }
    responses:
      "200": { description: Computed availability including exceptions and remaining capacity, content: {
application/json: { schema: { $ref: "#/components/schemas/Availability" } } } }
/public/qr/{reference}:
  get:
    operationId: resolveQr
    security: []
    parameters: [{ name: reference, in: path, required: true, schema: { type: string } }]
    responses:
      "200": { description: Active booking context without personal data, content: { application/json: {
schema: { $ref: "#/components/schemas/QrContext" } } } }
      "404": { $ref: "#/components/responses/Error" }
/clinics:
  get:
    operationId: listClinics
    parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/Status" }, {
name: city, in: query, schema: { type: string } }, { $ref: "#/components/parameters/Page" }, { $ref:
"#/components/parameters/PageSize" }, { $ref: "#/components/parameters/Sort" }]
    responses:
      "200": { description: Scoped clinics, content: { application/json: { schema: { $ref:
"#/components/schemas/ClinicList" } } } }
  post:
    operationId: createClinic
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/ClinicInput" } } } }
    responses:
      "201": { description: Created, content: { application/json: { schema: { $ref:
"#/components/schemas/Clinic" } } } }
/clinics/{id}:
  parameters: [{ $ref: "#/components/parameters/Id" }]
  get:
    operationId: getClinic
    responses:
      "200": { description: Clinic, content: { application/json: { schema: { $ref:
"#/components/schemas/Clinic" } } } }
  patch:
    operationId: updateClinic
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/ClinicInput" } } } }
    responses:
      "200": { description: Updated, content: { application/json: { schema: { $ref:
"#/components/schemas/Clinic" } } } }
  delete:
    operationId: deleteClinic
    responses:
      "204": { description: Deactivated }
/branches:
  get:
    operationId: listBranches
    parameters: [{ $ref: "#/components/parameters/ClinicId" }, { $ref: "#/components/parameters/Search" }, {
$ref: "#/components/parameters/Status" }, { $ref: "#/components/parameters/Page" }, { $ref:

```

```

    "#/components/parameters/PageSize" }, { $ref: "#/components/parameters/Sort" }]
    responses:
      "200": { description: Branches, content: { application/json: { schema: { $ref:
"#/components/schemas/BranchList" } } } }
    post:
      operationId: createBranch
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/BranchInput" } } } }
      responses:
        "201": { description: Created, content: { application/json: { schema: { $ref:
"#/components/schemas/Branch" } } } }
    /branches/{id}:
      parameters: [{ $ref: "#/components/parameters/Id" }]
      get:
        operationId: getBranch
        responses:
          "200": { description: Branch, content: { application/json: { schema: { $ref:
"#/components/schemas/Branch" } } } }
      patch:
        operationId: updateBranch
        requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/BranchInput" } } } }
        responses:
          "200": { description: Updated, content: { application/json: { schema: { $ref:
"#/components/schemas/Branch" } } } }
      delete:
        operationId: deleteBranch
        responses:
          "204": { description: Deactivated }
    /doctors:
      get:
        operationId: listDoctors
        parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { $ref: "#/components/parameters/Status" }, { name: specializationId,
in: query, schema: { type: string } }, { $ref: "#/components/parameters/Page" }, { $ref:
"#/components/parameters/PageSize" }, { $ref: "#/components/parameters/Sort" }]
        responses:
          "200": { description: Doctors only, content: { application/json: { schema: { $ref:
"#/components/schemas/DoctorList" } } } }
      post:
        operationId: createDoctor
        description: Authorized administrator creates doctor and identity invitation; never silently promotes an
unrelated existing patient.
        requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/DoctorInput" } } } }
        responses:
          "201": { description: Created, content: { application/json: { schema: { $ref:
"#/components/schemas/Doctor" } } } }
    /doctors/{id}:
      parameters: [{ $ref: "#/components/parameters/Id" }]
      get:
        operationId: getDoctor
        responses:
          "200": { description: Doctor, content: { application/json: { schema: { $ref:
"#/components/schemas/Doctor" } } } }
      patch:
        operationId: updateDoctor

```

```

        description: Own professional profile or authorized administration; clinic assignments are authorization-
checked.
        requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/DoctorInput" } } } }
        responses:
            "200": { description: Updated, content: { application/json: { schema: { $ref:
"#/components/schemas/Doctor" } } } }
        delete:
            operationId: deleteDoctor
            responses:
                "204": { description: Deactivated }
/users:
    get:
        operationId: listUsers
        parameters: [{ $ref: "#/components/parameters/Search" }, { name: role, in: query, schema: { $ref:
"#/components/schemas/Role" } }, { $ref: "#/components/parameters/ClinicId" }, { $ref:
"#/components/parameters/BranchId" }, { $ref: "#/components/parameters/Status" }, { $ref:
"#/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }, { $ref:
"#/components/parameters/Sort" }]
        responses:
            "200": { description: Authorized user administration, content: { application/json: { schema: { $ref:
"#/components/schemas/UserList" } } } }
    post:
        operationId: createUser
        description: Admin-only identity invitation and database role assignment; clinic admins cannot grant
platform administration.
        requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/UserInput" } } } }
        responses:
            "201": { description: Created, content: { application/json: { schema: { $ref: "#/components/schemas/User"
} } } }
/users/{id}:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    get:
        operationId: getUser
        responses:
            "200": { description: User, content: { application/json: { schema: { $ref: "#/components/schemas/User" }
} } }
    patch:
        operationId: updateUser
        requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/UserInput" } } } }
        responses:
            "200": { description: Updated, content: { application/json: { schema: { $ref: "#/components/schemas/User"
} } } }
    delete:
        operationId: deleteUser
        responses:
            "204": { description: Deactivated; last active super admin cannot be removed }
/users/{id}/password-reset:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    post:
        operationId: requestUserPasswordReset
        description: Requests Clerk-managed password recovery; never returns credentials.
        responses:
            "202": { description: Recovery requested, content: { application/json: { schema: { $ref:
"#/components/schemas/Message" } } } }

```

```

/patients:
  get:
    operationId: listPatients
    description: Patients visible only through authorized clinic registration or appointment relationship.
    parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { name: gender, in: query, schema: { type: string } }, { $ref:
"/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }, { $ref:
"/components/parameters/Sort" }]
    responses:
      "200": { description: Scoped patients, content: { application/json: { schema: { $ref:
"/components/schemas/PatientList" } } } }
  post:
    operationId: createPatient
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/PatientInput" } } } }
    responses:
      "201": { description: Registered, content: { application/json: { schema: { $ref:
"/components/schemas/Patient" } } } }
/patients/{id}:
  parameters: [{ $ref: "#/components/parameters/Id" }]
  get:
    operationId: getPatient
    responses:
      "200": { description: Patient, content: { application/json: { schema: { $ref:
"/components/schemas/Patient" } } } }
  patch:
    operationId: updatePatient
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/PatientInput" } } } }
    responses:
      "200": { description: Updated, content: { application/json: { schema: { $ref:
"/components/schemas/Patient" } } } }
  delete:
    operationId: deletePatient
    responses:
      "204": { description: Deactivated preserving medical visit history }
/masters:
  get:
    operationId: listMasters
    description: Authenticated dropdown reads; only super admins mutate. Workflow statuses and roles are
governed codes, not arbitrary new privileges.
    parameters: [{ name: category, in: query, schema: { type: string } }, { name: parentId, in: query, schema:
{ type: string } }, { $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/Status" }, {
$ref: "#/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }, { $ref:
"/components/parameters/Sort" }]
    responses:
      "200": { description: Dynamic masters, content: { application/json: { schema: { $ref:
"/components/schemas/MasterList" } } } }
  post:
    operationId: createMaster
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/MasterInput" } } } }
    responses:
      "201": { description: Created, content: { application/json: { schema: { $ref:
"/components/schemas/Master" } } } }
/masters/{id}:
  parameters: [{ $ref: "#/components/parameters/Id" }]

```

```

get:
  operationId: getMaster
  responses:
    "200": { description: Master, content: { application/json: { schema: { $ref:
"/components/schemas/Master" } } } }
  patch:
    operationId: updateMaster
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/MasterInput" } } } }
    responses:
      "200": { description: Updated, content: { application/json: { schema: { $ref:
"/components/schemas/Master" } } } }
  delete:
    operationId: deleteMaster
    responses:
      "204": { description: Deactivated }
/schedules:
get:
  operationId: listSchedules
  parameters: [{ $ref: "#/components/parameters/DoctorId" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { $ref: "#/components/parameters/Page" }, { $ref:
"/components/parameters/PageSize" }]
  responses:
    "200": { description: Weekly schedules, content: { application/json: { schema: { $ref:
"/components/schemas/ScheduleList" } } } }
  post:
    operationId: createSchedule
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/ScheduleInput" } } } }
    responses:
      "201": { description: Saved, content: { application/json: { schema: { $ref:
"/components/schemas/Schedule" } } } }
  /schedules/{id}:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    patch:
      operationId: updateSchedule
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/ScheduleInput" } } } }
      responses:
        "200": { description: Updated, content: { application/json: { schema: { $ref:
"/components/schemas/Schedule" } } } }
    delete:
      operationId: deleteSchedule
      responses:
        "204": { description: Removed without altering historical visits }
/availability-exceptions:
get:
  operationId: listAvailabilityExceptions
  parameters: [{ $ref: "#/components/parameters/DoctorId" }, { $ref: "#/components/parameters/BranchId" }, {
$ref: "#/components/parameters/From" }, { $ref: "#/components/parameters/To" }, { $ref:
"/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }]
  responses:
    "200": { description: Date overrides, content: { application/json: { schema: { $ref:
"/components/schemas/AvailabilityExceptionList" } } } }
  post:
    operationId: createAvailabilityException
    requestBody: { required: true, content: { application/json: { schema: { $ref:

```

```

"#/components/schemas/AvailabilityExceptionInput" } } }
  responses:
    "201": { description: Saved, content: { application/json: { schema: { $ref:
"#/components/schemas/AvailabilityException" } } } }
  /availability-exceptions/{id}:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    patch:
      operationId: updateAvailabilityException
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/AvailabilityExceptionInput" } } } }
    responses:
      "200": { description: Updated, content: { application/json: { schema: { $ref:
"#/components/schemas/AvailabilityException" } } } }
    delete:
      operationId: deleteAvailabilityException
      responses:
        "204": { description: Removed }
  /appointments:
    get:
      operationId: listAppointments
      parameters: [{ $ref: "#/components/parameters/Search" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { $ref: "#/components/parameters/DoctorId" }, { name: patientId, in:
query, schema: { type: string } }, { $ref: "#/components/parameters/Date" }, { $ref:
"#/components/parameters/From" }, { $ref: "#/components/parameters/To" }, { name: status, in: query, schema: {
$ref: "#/components/schemas/AppointmentStatus" } }, { name: source, in: query, schema: { $ref:
"#/components/schemas/BookingSource" } }, { $ref: "#/components/parameters/Page" }, { $ref:
"#/components/parameters/PageSize" }, { $ref: "#/components/parameters/Sort" }]
      responses:
        "200": { description: Scoped history and upcoming visits, content: { application/json: { schema: { $ref:
"#/components/schemas/AppointmentList" } } } }
    post:
      operationId: createAppointment
      description: Atomically checks assignment, computed availability, OTP policy and capacity, allocates
reference/token. Patients can book only self; staff may create walk-in and phone bookings. Duplicate requestId
returns original booking.
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"#/components/schemas/AppointmentInput" } } } }
      responses:
        "201": { description: Booking confirmation, content: { application/json: { schema: { $ref:
"#/components/schemas/Appointment" } } } }
        "409": { $ref: "#/components/responses/Error" }
  /appointments/{id}:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    get:
      operationId: getAppointment
      responses:
        "200": { description: Confirmation and lifecycle, content: { application/json: { schema: { $ref:
"#/components/schemas/Appointment" } } } }
  /appointments/{id}/actions:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    post:
      operationId: transitionAppointment
      description: |
        checkIn booked→checkedIn; enqueue checkedIn→waiting; call waiting→called;
        start called→inConsultation; complete inConsultation→completed;
        noShow waiting/called→noShow; requeue noShow→waiting;
        cancel booked/checkedIn/waiting→cancelled where policy allows.

```

```

    Patients may cancel only self. All other actions require scoped staff.
    Concurrent state conflicts return 409. Every transition is audited.
    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/AppointmentAction" } } } }
    responses:
      "200": { description: Updated lifecycle, content: { application/json: { schema: { $ref:
"/components/schemas/Appointment" } } } }
      "409": { $ref: "#/components/responses/Error" }
  /queue:
    get:
      operationId: getQueue
      description: Poll every 30 seconds. Patients get aggregate tokens/counts and own entry only; entries are
omitted for patients. Staff receive scoped identifiable entries.
      parameters:
        - { name: doctorId, in: query, required: true, schema: { type: string } }
        - { name: branchId, in: query, required: true, schema: { type: string } }
        - { name: date, in: query, required: true, schema: { type: string, format: date } }
        - { name: appointmentId, in: query, schema: { type: string } }
      responses:
        "200": { description: Live queue, content: { application/json: { schema: { $ref:
"/components/schemas/LiveQueue" } } } }
  /queue/call-next:
    post:
      operationId: callNext
      description: Scoped staff only. Locks doctor/branch/date queue; calls oldest waiting entry, rejecting
another active consultation. Empty queue returns appointment null.
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/QueueSelection" } } } }
      responses:
        "200": { description: Called next, content: { application/json: { schema: { $ref:
"/components/schemas/CallNextResult" } } } }
        "409": { $ref: "#/components/responses/Error" }
  /qrs:
    get:
      operationId: listQrs
      parameters: [{ $ref: "#/components/parameters/ClinicId" }, { $ref: "#/components/parameters/BranchId" }, {
$ref: "#/components/parameters/DoctorId" }, { $ref: "#/components/parameters/Status" }, { $ref:
"/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }]
      responses:
        "200": { description: Scoped QR configurations, content: { application/json: { schema: { $ref:
"/components/schemas/QrList" } } } }
    post:
      operationId: createQr
      requestBody: { required: true, content: { application/json: { schema: { $ref:
"/components/schemas/QrInput" } } } }
      responses:
        "201": { description: Secure public URL for QR rendering/download/print, content: { application/json: {
schema: { $ref: "#/components/schemas/Qr" } } } }
  /qrs/{id}:
    parameters: [{ $ref: "#/components/parameters/Id" }]
    get:
      operationId: getQr
      responses:
        "200": { description: QR, content: { application/json: { schema: { $ref: "#/components/schemas/Qr" } } } }
  }
  patch:
    operationId: updateQr

```

```

    requestBody: { required: true, content: { application/json: { schema: { $ref:
"/#/components/schemas/QrInput" } } } }
    responses:
      "200": { description: Updated, content: { application/json: { schema: { $ref: "#/components/schemas/Qr" }
} } }
    delete:
      operationId: deleteQr
      responses:
        "204": { description: Deactivated public reference }
/qrs/{id}/regenerate:
  parameters: [{ $ref: "#/components/parameters/Id" }]
  post:
    operationId: regenerateQr
    description: Revokes previous reference and generates a cryptographically random replacement.
    responses:
      "200": { description: New reference, content: { application/json: { schema: { $ref:
"/#/components/schemas/Qr" } } } }
/dashboard:
  get:
    operationId: getDashboard
    parameters: [{ $ref: "#/components/parameters/Date" }, { $ref: "#/components/parameters/ClinicId" }, {
$ref: "#/components/parameters/BranchId" }, { $ref: "#/components/parameters/DoctorId" }]
    responses:
      "200": { description: Role-scoped database aggregates, content: { application/json: { schema: { $ref:
"/#/components/schemas/Dashboard" } } } }
/reports:
  get:
    operationId: getReports
    parameters: [{ $ref: "#/components/parameters/From" }, { $ref: "#/components/parameters/To" }, { $ref:
"/#/components/parameters/ClinicId" }, { $ref: "#/components/parameters/BranchId" }, { $ref:
"/#/components/parameters/DoctorId" }, { name: groupBy, in: query, schema: { type: string, enum: [clinic, doctor,
date], default: date } }]
    responses:
      "200": { description: Scoped appointment/registration/queue performance report, content: {
application/json: { schema: { $ref: "#/components/schemas/Report" } } } }
/audit-logs:
  get:
    operationId: listAuditLogs
    description: Admin-only scoped summaries; sensitive old/new values are redacted.
    parameters: [{ $ref: "#/components/parameters/Search" }, { name: entityType, in: query, schema: { type:
string } }, { name: actorId, in: query, schema: { type: string } }, { $ref: "#/components/parameters/ClinicId" },
{ $ref: "#/components/parameters/From" }, { $ref: "#/components/parameters/To" }, { $ref:
"/#/components/parameters/Page" }, { $ref: "#/components/parameters/PageSize" }]
    responses:
      "200": { description: Immutable audit summaries, content: { application/json: { schema: { $ref:
"/#/components/schemas/AuditLogList" } } } }
/settings:
  get:
    operationId: getSettings
    description: Authenticated safe configuration only; provider credentials never returned.
    responses:
      "200": { description: Settings, content: { application/json: { schema: { $ref:
"/#/components/schemas/Settings" } } } }
  patch:
    operationId: updateSettings
    description: Super admin only.
    requestBody: { required: true, content: { application/json: { schema: { $ref:

```

```

"#/components/schemas/SettingsInput" } } } }
  responses:
    "200": { description: Saved, content: { application/json: { schema: { $ref:
"#/components/schemas/Settings" } } } }
components:
  securitySchemes:
    ClerkBearer: { type: http, scheme: bearer, bearerFormat: JWT }
  responses:
    Error:
      description: Request failed
      content: { application/json: { schema: { $ref: "#/components/schemas/ApiError" } } }
  parameters:
    Id: { name: id, in: path, required: true, schema: { type: string } }
    Search: { name: search, in: query, schema: { type: string } }
    Page: { name: page, in: query, schema: { type: integer, minimum: 1, default: 1 } }
    PageSize: { name: pageSize, in: query, schema: { type: integer, minimum: 1, maximum: 100, default: 20 } }
    Sort: { name: sort, in: query, description: Allowlisted field with optional minus prefix for descending
order, schema: { type: string } }
    Status: { name: status, in: query, schema: { type: string, enum: [active, inactive] } }
    ClinicId: { name: clinicId, in: query, schema: { type: string } }
    BranchId: { name: branchId, in: query, schema: { type: string } }
    DoctorId: { name: doctorId, in: query, schema: { type: string } }
    Date: { name: date, in: query, schema: { type: string, format: date } }
    From: { name: from, in: query, schema: { type: string, format: date } }
    To: { name: to, in: query, schema: { type: string, format: date } }
  schemas:
    HealthStatus:
      type: object
      required: [status]
      properties:
        status: { type: string }
    ApiError:
      type: object
      required: [error]
      properties:
        error: { type: string }
        code: { type: string }
    Message:
      type: object
      required: [message]
      properties:
        message: { type: string }
    Role: { type: string, enum: [superAdmin, clinicAdmin, doctor, receptionist, patient] }
    RecordStatus: { type: string, enum: [active, inactive] }
    AppointmentStatus: { type: string, enum: [booked, checkedIn, waiting, called, inConsultation, completed,
noShow, cancelled] }
    BookingSource: { type: string, enum: [online, walkIn, phone, qr] }
    PageMeta:
      type: object
      required: [total, page, pageSize]
      properties:
        total: { type: integer, minimum: 0 }
        page: { type: integer }
        pageSize: { type: integer }
    ProfileInput:
      type: object
      properties:

```

```

    fullName: { type: string, minLength: 1 }
    mobile: { type: string }
    photoUrl: { type: string }
OnboardingInput:
  type: object
  required: [fullName]
  properties:
    intent: { type: string, enum: [patient, doctor], default: patient }
    fullName: { type: string, minLength: 1 }
    mobile: { type: string }
    termsAccepted: { type: boolean }
Identity:
  type: object
  required: [clerkId, user, needsOnboarding]
  properties:
    clerkId: { type: string }
    user: { oneOf: [{ $ref: "#/components/schemas/User" }, { type: "null" }] }
    doctorId: { type: [string, "null"] }
    patientId: { type: [string, "null"] }
    needsOnboarding: { type: boolean }
UserInput:
  type: object
  required: [fullName, email, role]
  properties:
    fullName: { type: string, minLength: 1 }
    email: { type: string, format: email }
    mobile: { type: string }
    role: { $ref: "#/components/schemas/Role" }
    status: { $ref: "#/components/schemas/RecordStatus" }
    clinicIds: { type: array, items: { type: string } }
    branchIds: { type: array, items: { type: string } }
User:
  allOf:
    - { $ref: "#/components/schemas/UserInput" }
    - type: object
      required: [id, clerkId, status, clinicIds, branchIds, createdAt]
      properties:
        id: { type: string }
        clerkId: { type: [string, "null"] }
        createdAt: { type: string, format: date-time }
        lastLoginAt: { type: [string, "null"], format: date-time }
OtpRequest:
  type: object
  required: [mobile]
  properties:
    mobile: { type: string, pattern: '^\\+[1-9][0-9]{7,14}$' }
OtpChallenge:
  type: object
  required: [challengeId, expiresAt, resendAfterSeconds, provider]
  properties:
    challengeId: { type: string }
    expiresAt: { type: string, format: date-time }
    resendAfterSeconds: { type: integer }
    provider: { type: string, enum: [sms, development] }
    developmentCode: { type: string, description: Returned only by the explicitly enabled development
provider when NODE_ENV=development. Never present in production. }
OtpVerification:

```

```

    type: object
    required: [challengeId, code]
    properties:
      challengeId: { type: string }
      code: { type: string, pattern: '^[0-9]{4,8}$' }
OtpResult:
  type: object
  required: [verified, mobile]
  properties:
    verified: { type: boolean }
    mobile: { type: string }
    verifiedAt: { type: string, format: date-time }
ClinicInput:
  type: object
  required: [name, address]
  properties:
    name: { type: string, minLength: 1 }
    address: { type: string }
    country: { type: string }
    state: { type: string }
    city: { type: string }
    area: { type: string }
    pincode: { type: string }
    phone: { type: string }
    email: { type: string, format: email }
    description: { type: string }
    clinicTypeId: { type: string }
    categoryId: { type: string }
    status: { $ref: "#/components/schemas/RecordStatus" }
Clinic:
  allOf:
    - { $ref: "#/components/schemas/ClinicInput" }
    - type: object
      required: [id, code, status, createdAt]
      properties:
        id: { type: string }
        code: { type: string }
        createdAt: { type: string, format: date-time }
        doctorCount: { type: integer }
        branchCount: { type: integer }
BranchInput:
  type: object
  required: [clinicId, name, address]
  properties:
    clinicId: { type: string }
    name: { type: string, minLength: 1 }
    address: { type: string }
    city: { type: string }
    state: { type: string }
    pincode: { type: string }
    phone: { type: string }
    timezone: { type: string, default: Asia/Kolkata }
    status: { $ref: "#/components/schemas/RecordStatus" }
Branch:
  allOf:
    - { $ref: "#/components/schemas/BranchInput" }
    - type: object

```

```

      required: [id, code, status]
      properties:
        id: { type: string }
        code: { type: string }
        clinicName: { type: string }
DoctorInput:
  type: object
  required: [fullName, email]
  properties:
    fullName: { type: string, minLength: 1 }
    email: { type: string, format: email }
    mobile: { type: string }
    photoUrl: { type: string }
    gender: { type: string }
    dateOfBirth: { type: string, format: date }
    specializationId: { type: string }
    qualificationIds: { type: array, items: { type: string } }
    registrationNumber: { type: string }
    experienceYears: { type: integer, minimum: 0 }
    about: { type: string }
    consultationFee: { type: number, minimum: 0 }
    languages: { type: array, items: { type: string } }
    clinicIds: { type: array, items: { type: string } }
    branchIds: { type: array, items: { type: string } }
    status: { $ref: "#/components/schemas/RecordStatus" }
Doctor:
  allOf:
    - { $ref: "#/components/schemas/DoctorInput" }
    - type: object
      required: [id, userId, code, status]
      properties:
        id: { type: string }
        userId: { type: string }
        code: { type: string }
        specializationName: { type: string }
        qualificationNames: { type: array, items: { type: string } }
PublicDoctor:
  type: object
  required: [id, fullName, clinicIds, branchIds]
  properties:
    id: { type: string }
    fullName: { type: string }
    photoUrl: { type: string }
    specializationName: { type: string }
    qualificationNames: { type: array, items: { type: string } }
    about: { type: string }
    experienceYears: { type: integer }
    consultationFee: { type: number }
    clinicIds: { type: array, items: { type: string } }
    branchIds: { type: array, items: { type: string } }
PatientInput:
  type: object
  required: [fullName, mobile]
  properties:
    fullName: { type: string, minLength: 1 }
    mobile: { type: string }
    email: { type: string, format: email }

```

```

    dateOfBirth: { type: string, format: date }
    age: { type: integer, minimum: 0, maximum: 130 }
    gender: { type: string }
    address: { type: string }
    emergencyContactName: { type: string }
    emergencyContactPhone: { type: string }
    clinicId: { type: string }
    branchId: { type: string }
    status: { $ref: "#/components/schemas/RecordStatus" }
Patient:
  allOf:
    - { $ref: "#/components/schemas/PatientInput" }
    - type: object
      required: [id, code, createdAt, mobileVerified]
      properties:
        id: { type: string }
        code: { type: string }
        userId: { type: [string, "null"] }
        mobileVerified: { type: boolean }
        createdAt: { type: string, format: date-time }
MasterInput:
  type: object
  required: [category, name, code]
  properties:
    category: { type: string, enum: [country, state, city, area, pincode, clinicType, clinicCategory,
clinicStatus, specialization, qualification, department, consultationType, appointmentStatus, appointmentType,
bookingSource, cancellationReason, queueStatus, tokenPrefix, queueType, queuePriority, userRole, userStatus] }
    name: { type: string, minLength: 1 }
    code: { type: string, minLength: 1 }
    parentId: { type: [string, "null"] }
    sortOrder: { type: integer }
    status: { $ref: "#/components/schemas/RecordStatus" }
Master:
  allOf:
    - { $ref: "#/components/schemas/MasterInput" }
    - type: object
      required: [id, status]
      properties:
        id: { type: string }
ScheduleInput:
  type: object
  required: [doctorId, clinicId, branchId, dayOfWeek, isOpen, startTime, endTime, tokenPrefix, maxTokens,
consultationMinutes]
  properties:
    doctorId: { type: string }
    clinicId: { type: string }
    branchId: { type: string }
    dayOfWeek: { type: integer, minimum: 0, maximum: 6, description: Sunday is 0 }
    isOpen: { type: boolean }
    startTime: { type: string, pattern: '^[0-2][0-9]:[0-5][0-9]$' }
    endTime: { type: string }
    breakStart: { type: [string, "null"] }
    breakEnd: { type: [string, "null"] }
    timezone: { type: string, default: Asia/Kolkata }
    tokenPrefix: { type: string, minLength: 1, maxLength: 8 }
    maxTokens: { type: integer, minimum: 1 }
    consultationMinutes: { type: integer, minimum: 1 }

```

```

    bufferMinutes: { type: integer, minimum: 0, default: 0 }
    queueMode: { type: string, enum: [mixed, appointmentsOnly, walkInsOnly], default: mixed }
    queueOpenTime: { type: string }
    queueCloseTime: { type: string }
Schedule:
  allOf:
    - { $ref: "#/components/schemas/ScheduleInput" }
    - type: object
      required: [id]
      properties:
        id: { type: string }
        doctorName: { type: string }
        clinicName: { type: string }
        branchName: { type: string }
AvailabilityExceptionInput:
  type: object
  required: [doctorId, branchId, date, isClosed, reason]
  properties:
    doctorId: { type: string }
    branchId: { type: string }
    date: { type: string, format: date }
    isClosed: { type: boolean }
    reason: { type: string }
    startTime: { type: [string, "null"] }
    endTime: { type: [string, "null"] }
    breakStart: { type: [string, "null"] }
    breakEnd: { type: [string, "null"] }
    maxTokens: { type: [integer, "null"], minimum: 1 }
AvailabilityException:
  allOf:
    - { $ref: "#/components/schemas/AvailabilityExceptionInput" }
    - type: object
      required: [id]
      properties:
        id: { type: string }
Availability:
  type: object
  required: [doctorId, clinicId, branchId, date, available, maxTokens, bookedTokens, remainingTokens]
  properties:
    doctorId: { type: string }
    clinicId: { type: string }
    branchId: { type: string }
    date: { type: string, format: date }
    available: { type: boolean }
    reason: { type: [string, "null"] }
    startTime: { type: [string, "null"] }
    endTime: { type: [string, "null"] }
    breakStart: { type: [string, "null"] }
    breakEnd: { type: [string, "null"] }
    timezone: { type: string }
    maxTokens: { type: integer }
    bookedTokens: { type: integer }
    remainingTokens: { type: integer }
    consultationMinutes: { type: integer }
    tokenPrefix: { type: string }
AppointmentInput:
  type: object

```

```

    required: [patientId, doctorId, clinicId, branchId, date, source]
    properties:
      patientId: { type: string }
      doctorId: { type: string }
      clinicId: { type: string }
      branchId: { type: string }
      date: { type: string, format: date }
      source: { $ref: "#/components/schemas/BookingSource" }
      appointmentTypeId: { type: string }
      consultationTypeId: { type: string }
      qrReference: { type: string }
      requestId: { type: string, description: Client-generated idempotency key }
      termsAccepted: { type: boolean }
      notes: { type: string, maxLength: 1000 }
  Appointment:
    allOf:
      - { $ref: "#/components/schemas/AppointmentInput" }
      - type: object
        required: [id, reference, token, status, createdAt, patientName, doctorName, clinicName, branchName,
allowedActions]
        properties:
          id: { type: string }
          reference: { type: string }
          token: { type: string }
          tokenNumber: { type: integer }
          status: { $ref: "#/components/schemas/AppointmentStatus" }
          patientName: { type: string }
          patientCode: { type: string }
          doctorName: { type: string }
          clinicName: { type: string }
          branchName: { type: string }
          startTime: { type: string }
          endTime: { type: string }
          createdAt: { type: string, format: date-time }
          checkedInAt: { type: [string, "null"], format: date-time }
          calledAt: { type: [string, "null"], format: date-time }
          consultationStartedAt: { type: [string, "null"], format: date-time }
          completedAt: { type: [string, "null"], format: date-time }
          allowedActions: { type: array, items: { $ref: "#/components/schemas/AppointmentActionType" } }
          history: { type: array, items: { $ref: "#/components/schemas/StatusEvent" } }
  AppointmentActionType: { type: string, enum: [checkIn, enqueue, call, start, complete, noShow, requeue,
cancel] }
  AppointmentAction:
    type: object
    required: [action]
    properties:
      action: { $ref: "#/components/schemas/AppointmentActionType" }
      reason: { type: string }
      cancellationReasonId: { type: string }
      expectedStatus: { $ref: "#/components/schemas/AppointmentStatus" }
  StatusEvent:
    type: object
    required: [status, occurredAt]
    properties:
      status: { $ref: "#/components/schemas/AppointmentStatus" }
      occurredAt: { type: string, format: date-time }
      reason: { type: string }

```

```

QueueSelection:
  type: object
  required: [doctorId, branchId, date]
  properties:
    doctorId: { type: string }
    branchId: { type: string }
    date: { type: string, format: date }
OwnQueueEntry:
  type: object
  required: [appointmentId, token, status, patientsAhead, estimatedWaitMinutes]
  properties:
    appointmentId: { type: string }
    token: { type: string }
    status: { $ref: "#/components/schemas/AppointmentStatus" }
    patientsAhead: { type: integer }
    estimatedWaitMinutes: { type: integer }
LiveQueue:
  type: object
  required: [doctorId, branchId, date, currentToken, nextToken, waiting, inConsultation, completed, noShow,
total, pollIntervalSeconds, updatedAt]
  properties:
    doctorId: { type: string }
    branchId: { type: string }
    date: { type: string, format: date }
    currentToken: { type: [string, "null"] }
    nextToken: { type: [string, "null"] }
    waiting: { type: integer }
    inConsultation: { type: integer }
    completed: { type: integer }
    noShow: { type: integer }
    total: { type: integer }
    ownEntry: { oneOf: [{ $ref: "#/components/schemas/OwnQueueEntry" }, { type: "null" }] }
    entries: { type: array, description: Omitted for patients, items: { $ref:
"/components/schemas/Appointment" } }
    pollIntervalSeconds: { type: integer, const: 30 }
    updatedAt: { type: string, format: date-time }
CallNextResult:
  type: object
  required: [appointment]
  properties:
    appointment: { oneOf: [{ $ref: "#/components/schemas/Appointment" }, { type: "null" }] }
QrInput:
  type: object
  required: [name, clinicId]
  properties:
    name: { type: string }
    clinicId: { type: string }
    branchId: { type: [string, "null"] }
    doctorId: { type: [string, "null"] }
    status: { $ref: "#/components/schemas/RecordStatus" }
Qr:
  allOf:
    - { $ref: "#/components/schemas/QrInput" }
    - type: object
      required: [id, reference, bookingUrl, status, createdAt]
      properties:
        id: { type: string }

```

```

        reference: { type: string }
        bookingUrl: { type: string }
        createdAt: { type: string, format: date-time }
    }
    QrContext:
        type: object
        required: [reference, clinicId, clinicName]
        properties:
            reference: { type: string }
            clinicId: { type: string }
            clinicName: { type: string }
            branchId: { type: [string, "null"] }
            branchName: { type: [string, "null"] }
            doctorId: { type: [string, "null"] }
            doctorName: { type: [string, "null"] }
    Dashboard:
        type: object
        required: [todayAppointments, waiting, completed, noShow, checkedIn, cancelled, averageWaitMinutes]
        properties:
            totalDoctors: { type: integer }
            totalClinics: { type: integer }
            totalBranches: { type: integer }
            totalPatients: { type: integer }
            todayAppointments: { type: integer }
            activeQueues: { type: integer }
            waiting: { type: integer }
            checkedIn: { type: integer }
            completed: { type: integer }
            cancelled: { type: integer }
            noShow: { type: integer }
            averageWaitMinutes: { type: number }
            currentToken: { type: [string, "null"] }
            recentAppointments: { type: array, items: { $ref: "#/components/schemas/Appointment" } }
            recentActivity: { type: array, items: { $ref: "#/components/schemas/AuditLog" } }
    ReportRow:
        type: object
        required: [key, label, appointments, completed, cancelled, noShow, registrations, averageWaitMinutes]
        properties:
            key: { type: string }
            label: { type: string }
            appointments: { type: integer }
            checkedIn: { type: integer }
            waiting: { type: integer }
            completed: { type: integer }
            cancelled: { type: integer }
            noShow: { type: integer }
            registrations: { type: integer }
            averageWaitMinutes: { type: number }
            averageConsultationMinutes: { type: number }
    Report:
        type: object
        required: [from, to, groupBy, rows]
        properties:
            from: { type: string, format: date }
            to: { type: string, format: date }
            groupBy: { type: string }
            rows: { type: array, items: { $ref: "#/components/schemas/ReportRow" } }
    AuditLog:

```

```

type: object
required: [id, actorId, action, entityType, entityId, summary, createdAt]
properties:
  id: { type: string }
  actorId: { type: string }
  actorName: { type: string }
  actorRole: { $ref: "#/components/schemas/Role" }
  action: { type: string }
  entityType: { type: string }
  entityId: { type: string }
  clinicId: { type: [string, "null"] }
  summary: { type: string }
  createdAt: { type: string, format: date-time }
SettingsInput:
  type: object
  properties:
    platformName: { type: string }
    supportEmail: { type: string, format: email }
    supportPhone: { type: string }
    timezone: { type: string }
    bookingHorizonDays: { type: integer, minimum: 1 }
    cancellationCutoffMinutes: { type: integer, minimum: 0 }
    requireMobileVerification: { type: boolean }
    otpExpirySeconds: { type: integer, minimum: 60, maximum: 900 }
    otpMaxAttempts: { type: integer, minimum: 1, maximum: 10 }
    sessionTimeoutMinutes: { type: integer, minimum: 5 }
    notificationsEnabled: { type: boolean }
    logoUrl: { type: string }
    primaryColor: { type: string }
    termsUrl: { type: string }
    privacyUrl: { type: string }
Settings:
  allOf:
    - { $ref: "#/components/schemas/SettingsInput" }
    - type: object
      required: [platformName, requireMobileVerification, otpProviderConfigured, queuePollSeconds]
      properties:
        otpProviderConfigured: { type: boolean }
        queuePollSeconds: { type: integer, const: 30 }
ClinicList:
  allOf:
    - { $ref: "#/components/schemas/PageMeta" }
    - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Clinic" } } } }
BranchList:
  allOf:
    - { $ref: "#/components/schemas/PageMeta" }
    - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Branch" } } } }
DoctorList:
  allOf:
    - { $ref: "#/components/schemas/PageMeta" }
    - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Doctor" } } } }
PublicDoctorList:
  allOf:
    - { $ref: "#/components/schemas/PageMeta" }

```

```

      - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/PublicDoctor" } } } }
    UserList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/User" } } } }
    PatientList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Patient" } } } }
    MasterList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Master" } } } }
    ScheduleList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Schedule" } } } }
    AvailabilityExceptionList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/AvailabilityException" } } } }
    AppointmentList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Appointment" } } } }
    QrList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/Qr" } } } }
    AuditLogList:
      allOf:
        - { $ref: "#/components/schemas/PageMeta" }
        - { type: object, required: [items], properties: { items: { type: array, items: { $ref:
"#/components/schemas/AuditLog" } } } }

```

Generated from version-controlled documentation and the API contract. No external assets, analytics, credentials, or application connections are required to read this file.